교원 연수 교재

클로드 코드 기반 하네스 엔지니어링

Claude Code로 반복 업무를 맡기는 팀을 설계하고, Firebase 기능 지도를 익힌다

연수 시간	180분 (3시간)
구성	7개 모듈 + 마무리, 슬라이드 71장
판형	A4 세로 인쇄용 교재판
제작	대구과학고등학교 교사 이준구

소속 _____ 이름 _____

이 교재를 쓰는 법

슬라이드 한 장이 교재 한 항목입니다. 항목은 아래 순서로 읽습니다.

1. **본문** 슬라이드에 나오는 내용
2. **도식 · 단계** 콘솔·화면에서 따라 할 순서
3. **프롬프트 · 파일** Claude Code에 입력하거나 저장할 문구
4. **예상 결과** 성공했을 때 보이는 모습
5. **막힐 때** 자주 막히는 곳과 해결
6. **해설** 발표자 노트. 왜 그렇게 하는지 설명

표시 읽는 법

개념	설명을 듣고 이해하는 쪽
실습	직접 손을 움직이는 쪽. 체크 칸과 메모 줄이 있습니다.
점검	스스로 답해 보는 질문 쪽
정리	한 모듈을 마무리하는 쪽
확인 필요	콘솔 화면·메뉴 이름처럼 연수 전에 실제 화면으로 다시 확인해야 하는 내용
설문 링크/QR	점선 상자는 강사가 연수 전에 링크를 채우는 자리

상자 읽는 법

붙여 넣어 쓰는 프롬프트는 이렇게 왼쪽 붉은 선이 있는 상자에 담깁니다.

파일 이름.md

저장할 파일의 내용은 왼쪽 청록색 선이 있는 상자에 담깁니다.

목차

모듈 M1 도입 · Claude Code 첫 대화

1 Claude Code로 배우는 하네스 엔지니어링
2 오늘의 지도
3 채팅 AI와 Claude Code는 무엇이 다른가
4 터미널을 열고 설치 확인하기
5 Windows 설치가 막히면: Docker로 쓰는 대안
6 Docker 이미지 만들기: 빌드 (해당자만)
7 Docker로 Claude Code 시작하기 (해당자만)
8 실습 폴더에서 첫 대화
9 막히면 이렇게

모듈 M2 잘 시키는 법: 프롬프트 · 컨텍스트 · 플랜 모드

10 모호한 요청 vs 구체적인 요청
11 컨텍스트와 플랜 모드
12 실습: 플랜 모드로 수업 개요 만들기
13 실습: 파일 확인하고 이어서 고치기
14 방금 무슨 일이 있었나

모듈 M3 CLAUDE.md · 스킬 · 권한

15 CLAUDE.md = 세션 시작 시 자동 로드되는 프로젝트 지침
16 실습: /init 으로 CLAUDE.md 만들고 규칙 5개 추가
17 실습: 규칙이 지켜지는지 확인하기
18 슬래시 커맨드와 스킬 = 저장된 절차서
19 실습: /quiz 스킬 만들기
20 실습: /quiz 로 주제만 바꿔 재사용
21 권한 모드: 도구 실행 승인 정책
22 점검: Track 1 체크포인트 6문장

모듈 M4 하네스 입문

23 하네스가 해 주는 일
24 실습: 플러그인 설치 전 준비 (새 폴더에서 시작)
25 실습: 플러그인 설치 (설치 통로 열고 설치하기)
26 실습: 말로 팀 만들기 (플랜 모드)
27 실습: 팀 실행하고 검수 결과 보기
28 정리: 하네스가 만든 것

모듈 M5 하네스가 만든 파일 읽고 고치기

29 하네스가 만든 폴더 지도
30 실습: 생성된 파일 열어 보고 구조 설명받기
31 실습: 말로 고치기
32 실습: 고친 팀 다시 실행해 비교하기
33 실습: 내 업무용 새 팀을 하네스에게 요청하기
34 점검: 하네스가 만든 것, 사람이 확인할 것
35 정리: 하네스로 만들고, 말로 고치고, 다시 실행
36 휴식 5분

모듈 M6 Firebase 기능 지도와 콘솔 설정

37 Firebase 기능 지도: 앱 백엔드 서비스 묶음
38 공통 설정 1: 테스트 프로젝트 만들기
39 공통 설정 2: 웹 앱 등록과 firebaseConfig
40 Authentication 사용 상황: 사용자 인증과 uid
41 Authentication 설정 절차: Google 로그인 켜기

42 Firestore 사용 상황: 문서 지향 NoSQL 데이터베이스
43 Firestore 설정 절차: 데이터베이스 만들기
44 요금제 사용 상황: Spark와 Blaze
45 요금제 설정 절차: Blaze로 바꾸고 예산 알림 걸기
46 Storage 사용 상황: 파일(객체) 저장
47 질문: Firestore냐, Storage냐, 올리지 않을까
48 Storage 설정 절차: 버킷 만들기
49 Hosting 사용 상황: 정적 웹 콘텐츠 호스팅
50 Hosting 설정 절차: Node.js·CLI 설치와 로그인
51 질문: 배포 버튼을 누르기 전, 사람이 꼭 볼 것 3가지는?
52 Cloud Functions 사용 상황: 서버리스 백엔드 코드
53 Cloud Functions 설정 절차: 함수 배포와 비밀값
54 종합 상황 카드: 필요한 기능과 요금제는?
55 설정 체크리스트와 정리

모듈 M7 프로젝트: 우리 교과 단원 수업 패키지 만들기

56 오늘의 프로젝트: 우리 교과 단원 수업 패키지
57 단원 명세 쓰기: unit-spec.md
58 하네스에게 팀 구성 요청하기
59 수업 슬라이드 만들기: 하네스 팀 실행 1단계
60 사람이 확인: 교과 내용과 학습 목표
61 터미널에서 한 번 로그인하면 Claude Code가 이어받는다
62 Firebase 연결: 미리 로그인해 두기
63 워크북 웹앱 만들고 배포하기
64 배포 주소로 확인하기
65 고쳐 달라고 말하고 다시 배포하기
66 짝과 바꿔 점검하기
67 오늘 만든 패키지 한 장 회고

마무리

68 오늘 해낸 일 체크리스트
69 흔한 안티패턴 2가지
70 자주 묻는 질문 4가지
71 참고 자료와 다음 걸음

MODULE M1

모듈 M1

도입 · Claude Code 첫 대화

학습 목표

> Claude Code가 채팅 AI와 무엇이 다른지 설명하고, 실습 폴더에서 첫 대화를 할 수 있다.

시간	**13분**
슬라이드	**1-9 (9장)**
실습 산출물	**실습 폴더에서 실행한 Claude Code 첫 대화 화면**

이 모듈의 슬라이드

번호	제목	유형	시간	쪽
1	Claude Code로 배우는 하네스 엔지니어링	정리	1분	·
2	오늘의 지도	개념	1분	·
3	채팅 AI와 Claude Code는 무엇이 다른가	개념	2분	·
4	터미널을 열고 설치 확인하기	실습	2분	·
5	Windows 설치가 막히면: Docker로 쓰는 대안	개념	1분	·
6	Docker 이미지 만들기: 빌드 (해당자만)	실습	1분	·
7	Docker로 Claude Code 시작하기 (해당자만)	실습	1분	·
8	실습 폴더에서 첫 대화	실습	3분	·
9	막히면 이렇게	정리	1분	·

1 슬라이드 1 / 71

정리 1분

Claude Code로 배우는 하네스 엔지니어링

클로드 코드 기반 하네스 엔지니어링 교원 연수 (180분)

오늘 가져갈 것 3개: ① 하네스로 만든 팀을 쓰고 말로 고치는 경험(나의 Claude Code 작업 폴더) ② 우리 교과 단원 수업 패키지(슬라이드와 학생 워크북)를 Firebase Hosting 주소로 배포·확인한 결과 ③ Firebase 기능 지도

해설 · 발표자 노트

소요 시간 1분

환영 인사. "오늘은 앱을 만드는 날이 아니라, 에이전트형 코딩 도구를 잘 쓰는 방법과 하네스 플러그인으로 에이전트 팀을 만들어 쓰고 고치는 방법을 배우는 날입니다." 오늘 끝나면 손에 남는 것 세 가지를 짚는다. 원본 intro의 목표(Track 1: 혼자 쓰기, Track 2: 하네스 활용)를 한 문장으로 요약해 말한다. 이 연수는 멀티 에이전트의 원리를 배우는 것이 아니라 하네스를 활용하는 연수라고 한 줄로 말한다. 원본 대비 변경: 원본의 책·오픈소스 소개 문단은 슬라이드 71(참고 자료)로 넘긴다. (1분)

도식 설명 표지. 제목 아래에 "가져갈 것" 카드 3장을 가로로 나란히(폴더 아이콘 / 배포 주소 아이콘 / 지도 아이콘).

---

2 슬라이드 2 / 71

개념 1분 확인 필요

오늘의 지도

앱은 예시 소재일 뿐, 배우는 것은 방법입니다.

준비물 점검: 유료 Claude 계정 · 개인 구글 계정 · Git(하네스 설치에) · 결제 수단(Blaze 요금제에) · 실제 학생 정보는 쓰지 않기

도식 · 단계

Claude Code 기본과 하네스 활용 (M1-M5)

M1 첫 대화 → M2 프롬프트 → M3 CLAUDE.md·스킬 → M4 하네스 입문 → M5 하네스 파일 고치기

쉬는 시간 10분 후, Firebase와 프로젝트 (M6-M7)

M6 Firebase 지도 → M7 수업 패키지 프로젝트

해설 · 발표자 노트

소요 시간 1분

로드맵을 한 번만 훑는다. 앞 절반은 "혼자 쓰는 법과 하네스로 팀 만들기·고치기", 뒤 절반은 "Firebase 기능 지도와 수업 패키지 프로젝트". 마지막 M7은 50분 동안 각자 자기 교과 단원의 슬라이드와 학생 워크북을 만드는 프로젝트이며, 전체는 180분이다. 클래스룸 앱은 "예를 들면"으로만 나온다고 못박는다. 사전 준비 점검(손들기): ① Claude Code는 Pro·Max·Team·Enterprise 또는 Console(API) 계정이 필요하고 무료 claude.ai 계정으로는 쓸 수 없다 ② Firebase 실습용 개인 구글 계정(학교 관리 계정은 프로젝트 생성이 막혀 있을 수 있음 확인 필요) ③ 실제 학생 이름·계정은 쓰지 않는다 ④ Git: Claude Code 설치에는 필요 없지만 하네스 플러그인 설치(M4 25번)에는 필수다. `git --version`으로 확인하고, Windows는 없으면 Git for Windows를 설치한다(슬라이드 4) ⑤ Node.js·Firebase CLI는 M6 50번에서 함께 설치하고 로그인한다. 미리 설치해 두면 시간이 남는다. 없으면 50번 막힐 때를 따른다 확인 필요. ⑥ 결제 수단: M6 45번에서 Firebase 테스트 프로젝트를 Blaze 요금제로 바꾸고 Storage·Cloud Functions를 함께 해 본다. Blaze는 결제 수단 등록이 필수이므로 **본인 개인 계정에 본인 결제 수단**을 준비해 오게 한다(학교 카드·타인 카드 금지). 무료 사용량 안에서는 비용이 0일 수 있으나 예산 알림을 먼저 걸고, 연수 후 테스트 프로젝트를 삭제한다. 결제 수단을 쓰고 싶지 않은 참가자는 짝 화면을 보며 따라간다. 학교 관리 PC는 관리자 권한 설치 제한이 있을 수 있다(원본 intro). ⑦ Windows에서 설치 명령이 막히면: Docker Desktop을 쓰는 대안 경로(슬라이드 5~7)가 있다. Docker Desktop 설치(약 10~15분, WSL 2·가상화 필요)는 연수 시간에 하기에 길어서 사전 과제로 안내한다 확인 필요. OS 요구사항: macOS 13.0 이상, Windows 10(1809) 이상. 원본 대비 변경: 원본의 "3.5~4시간, Track 1·2" 구성은 180분 7개 모듈(총 71장)로 재구성했다. (1분)

도식 설명 `.flow` 가로 도식, 8칸(휴식 포함). 첫 다섯 칸은 Claude Code 기본(M1 첫 대화 → M2 프롬프트 → M3 CLAUDE.md·스킬 → M4 하네스 입문 → M5 하네스가 만든 파일 고치기), 휴식, 이어서 M6 Firebase 지도(27분) → M7 수업 패키지 프로젝트(50분). 현재 위치(M1)를 강조색으로 표시.

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 상세 없음
- 학교 PC 설치 권한
- 학교 PC 설치 권한·가상화

3

슬라이드 3 / 71

채팅 AI와 Claude Code는 무엇이 다른가

개념 2분

채팅 AI는 답을 주고, Claude Code는 직접 해 봅니다.
Claude Code는 터미널에서 동작하는 에이전트형 코딩 도구로, 파일 읽기·수정과 명령 실행을 도구 호출로 수행합니다.
(에이전틱 루프 = 모델이 도구를 호출하고, 결과를 관찰해 다음 행동을 정하는 반복)

해설 · 발표자 노트

소요 시간 2분

채팅 AI는 텍스트로 답을 돌려주고, Claude Code는 도구 호출로 작업을 직접 수행한다. 채팅은 결과를 내가 저장·실행하고 오류도 내가 다시 붙여넣어야 한다. Claude Code는 내 컴퓨터의 실제 폴더에서 파일을 읽고 쓰고 명령을 실행하며 오류까지 스스로 확인한다. 사람은 결과를 검토하고 승인하는 역할이다. 이 루프를 에이전틱 루프라고 부른다. 반드시 말할 것: "이름은 코드지만 수업 자료(HTML, 문항)를 만드는 데 쓴다." 예: 만든 HTML 파일의 문법 오류를 명령줄에서 검사하게 시킬 수 있다. 터미널 CLI 외에 데스크톱 앱, VS Code·JetBrains 확장, 웹(claude.ai/code)도 있으나 오늘은 터미널 CLI 기준이다(한 번 익히면 다른 형태로 넘어가기 쉽다). (2분)

도식 설명 카드 2장 비교. 왼쪽 "일반 채팅 AI: 묻고, 복사하고, 붙여넣는다"(회색 태그), 오른쪽 "Claude Code: 직접 읽고, 쓰고, 실행한다"(초록 `good` 태그). 아래에 순환 화살표 한 줄: 해 보기 → 결과 보기 → 다음 행동.

4

슬라이드 4 / 71

터미널을 열고 설치 확인하기

실습 2분 확인 필요

① 터미널 열기: macOS는 `⌘+Space`에서 "터미널", Windows는 시작 메뉴에서 "PowerShell"
② 아래 명령으로 버전이 보이면 이미 설치된 것입니다.
(터미널 = 말로 시키는 대신 글자로 명령하는 창)
③ Git도 필요합니다. Claude Code 설치에는 필요 없지만, 하네스 플러그인 설치(M4)에 필수입니다. `git --version`에 버전이 보이면 준비 완료. Windows에서 없으면 Git for Windows를 먼저 설치합니다.

프롬프트 · 파일

(터미널에 입력. Claude Code에 붙여넣는 문장이 아니라 터미널 명령)

```
claude --version
git --version
```

미설치자용 설치 명령(OS 탭):

- macOS / Linux: `curl -fsSL https://claude.ai/install.sh | bash`
- Windows PowerShell: `irm https://claude.ai/install.ps1 | iex`

- Windows CMD: `curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del install.cmd`

예상 결과

```
$ claude --version
2.1.211 (Claude Code)
$ git --version
git version 2.45.0
```

`2.1.211`과 `2.45.0`은 예시 숫자입니다. 내 화면의 번호는 달라도 됩니다. 두 명령 모두 버전 번호가 보이면 성공입니다.

막힐 때

- `command not found` → 터미널을 완전히 닫았다가 다시 엽니다.
- `git`을 찾을 수 없다고 나옴 → Windows는 Git for Windows(git-scm.com/downloads/win)를 설치하고 터미널을 새로 엽니다. 설치하지 않으면 M4 25번 플러그인 설치에서 `Failed to clone marketplace repository: Command 'git' not found`로 막힙니다.
- Windows에서 `irm`이 "인식할 수 없는 명령" → CMD입니다(프롬프트가 `C:\>`). PowerShell(`PS C:\>`)에서 PowerShell 탭 명령을 씁니다.
- 설치 명령이 계속 오류(`403`, TLS, PATH 등) → 다음 슬라이드 5~7의 Docker 경로(이미지 만들기와 실행)로 바꿉니다.

해설 · 발표자 노트

소요 시간 2분

이미 설치한 분이 대부분이므로 `claude --version`만 확인시키고 끝낸다. 미설치자는 OS 탭에서 자기 명령을 복사해 붙여넣고 Enter. Git은 Claude Code 설치에는 필요 없지만, 이 연수의 하네스 플러그인 설치(M4 25번의 `/plugin marketplace add revfactory/harness`)에는 필수다. 마켓플레이스를 GitHub 저장소에서 git으로 내려받기 때문이다. 공식 문서(plugin 문서와 troubleshooting, 2026-10-25 확인)에 따르면 Windows에서 git이 없으면 마켓플레이스 등록이 `Failed to clone marketplace repository: Command 'git' not found or is in an unsafe location`로 실패하고, 해결은 Git for Windows(https://git-scm.com/downloads/win) 설치 후 다시 시도하는 것이다. 그래서 M1에서 `git --version`으로 미리 확인시킨다. macOS·Linux도 같은 명령으로 확인하고, 없으면 설치 방법을 Claude Code나 강사에게 묻는다 확인 필요. Git for Windows가 있으면 Claude Code가 Git Bash로 Bash 도구를 쓸 수 있다(없으면 PowerShell 도구로 대신). 학교 PC에서 설치 권한이 없으면 짝 화면을 함께 보거나 강사 화면을 따라 본다. macOS는 공식 설치 스크립트(네이티브 설치)를 권장하며 Node.js를 먼저 깔 필요가 없다(원본). 프롬프트가 `PS`로 시작하면 PowerShell, `C:\>`만 보이면 CMD. 설치 명령 3종은 원본 그대로 옮겼고 공식 setup 문서와 일치한다(검수 확인). 학교 관리 PC에서 설치가 막히면 짝 화면 함께 보기, 강사 화면 따라 보기로 전환한다. 원본 대비 변경: 원본 1~3단계를 한 장으로 합쳤고 1단계 터미널 열기는 본문으로 압축. 설치 오류가 계속되는 Windows 참가자는 여기서 멈추지 말고 슬라이드 5~7의 Docker 경로로 안내한다. (2분)

도식 설명 OS 탭 없이 터미널 화면 예시 1장(커서 깜빡이는 창). 아래에 "미설치자용 설치 명령" `.code-block` + OS 탭 3개(macOS / Linux, Windows PowerShell, Windows CMD).

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- OS별 git 기본 설치 여부

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

---

5 슬라이드 5 / 71

개념 1분 확인 필요

Windows 설치가 막히면: Docker로 쓰는 대안

설치 스크립트 대신, 작은 리눅스 환경(컨테이너) 안에서 Claude Code를 실행합니다.
Dockerfile로 이미지를 한 번 만들면(빌드) Claude Code·Git·Node.js·Firebase CLI가 들어 있어 설치 오류, PATH, Git Bash 문제가 생길 자리가 없습니다.
대신 Docker Desktop 설치가 필요합니다. 4번 설치가 막힌 Windows 참가자만 6~7번을 따라 하고, 이미 설치한 분은 8번으로 갑니다.
단계별 설명서: dshs-gemini.github.io/harness-workshop/docker

해설 · 발표자 노트

소요 시간 1분

4번에서 설치가 막힌 Windows 참가자를 위한 대안 경로다. 나머지 참가자는 8번으로 바로 간다. 핵심 한 문장: "Docker는 설치 문제를 없애는 것이 아니라 'Docker Desktop 설치 한 번'으로 모아 준다." 컨테이너는 내 PC 안의 작은 리눅스 환경이고, Claude Code는 그 안에서 실행된다. 내 PC에서는 지정한 폴더(`work`) 하나만 공유되며, Claude Code가 만든 파일은 그 폴더에 그대로 나타난다. 이 경로는 두 단계다: 이미지를 한 번 만들고(빌드, 6번), 그 이미지로 Claude Code를 실행한다(7번). 이미지에 Git이 들어 있으므로 M4 25번의 하네스 플러그인 설치(`/plugin marketplace add`)도 Git for Windows 없이 된다. 한계: ① Docker Desktop은 Windows 10 22H2(빌드 19045) 이상 또는 Windows 11 23H2(빌드 22631) 이상, RAM 8GB, BIOS 가상화 사용이 필요하다(Docker 공식 문서). 가상화가 꺼져 있고 바꿀 수 없는 학교 PC에서는 Docker Desktop도 설치되지 않으므로 이때는 짝 화면을 함께 보거나 Claude 데스크톱 앱·웹 버전을 쓴다. ② Docker Desktop은 교육 목적에는 무료이지만 정부 기관은 유료 구독이 필요하다고 Docker 문서에 적혀 있어, 소속 기관 정책 확인이 필요하다 확인 필요. ③ 학교 네트워크가 Docker Hub·npm·claude.ai를 막으면 똑같이 실패한다. Docker Hub는 로그인 없이 IP당 6시간에 100회까지만 내려받을 수 있어, 여러 명이 동시에 빌드하면 한도에 걸릴 수 있다. 그래서 빌드가 막히면 강사가 만든 이미지 파일(`claude-workshop.tar`)로 `docker load`를 쓴다(6번 막힐 때). 사전 준비: Docker Desktop 설치는 연수 전 과제로 안내한다. 이 경로의 파일(Dockerfile·compose.yaml)과 단계별 설명은 보충 자료 "Docker 사용 가이드"(https://dshs-gemini.github.io/harness-workshop/docker/)와 저장소의 `docker-kit/` 폴더에 있다. 슬라이드에서 링크를 누르면 열린다. 원본 대비 변경: 원본 HTML에 없는 내용으로, 공식 문서(Claude Code devcontainer·sandbox-environments·setup, Docker Desktop for Windows)와 macOS에서의 빌드 확인을 근거로 추가했다. Windows 실기 확인은 아직 하지 못했다 확인 필요. (1분)

도식 설명 2열 카드. 왼쪽 `good` "사라지는 문제": 설치 명령 오류(`irm`·`403`·TLS), `claude`를 못 찾음(PATH), Git for Windows 설치, Node.js·Firebase CLI 설치. 오른쪽 "새로 생기는 조건": Docker Desktop 설치(WSL 2·가상화), 디스크 약 3GB와 RAM 8GB 이상, 학교 네트워크 차단은 그대로. 아래에 가로 한 줄: 내 PC의 `work` 폴더 ⇄ 컨테이너의 `/workspace` (이 폴더만 공유).

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 상세 없음

6 슬라이드 6 / 71

Docker 이미지 만들기: 빌드 (해당자만)

실습 1분 확인 필요

① Docker Desktop을 켜고 'Engine running'이 보일 때까지 기다립니다.

② 강사가 준 `docker-kit` 폴더(Dockerfile, compose.yaml, work)를 `C:\claude-docker`에 두고, 그 폴더에서 PowerShell을 엽니다.

③ `docker compose build`를 실행합니다. Dockerfile대로 이미지를 만드는 과정이며, 처음 한 번만 합니다. 몇 분 걸립니다.

④ 마지막에 `Image claude-workshop Built`가 보이면 완료입니다.

프롬프트 · 파일

(PowerShell에 입력)

```
cd C:\claude-docker
docker compose build
docker images
```

강사에게 이미지 파일(`claude-workshop.tar`)을 받았다면 빌드 대신 아래 명령으로 불러옵니다(빌드를 건너뜁니다).

```
docker load -i claude-workshop.tar
```

예상 결과

```
Image claude-workshop Built
REPOSITORY          TAG       SIZE
claude-workshop     latest    1.8GB
```

크기와 진행 로그는 PC마다 다를 수 있습니다. 중간에 진행 표시가 오래 멈춘 것처럼 보여도 정상일 수 있으니 몇 분은 기다립니다. 막히면 설명서: dshs-gemini.github.io/harness-workshop/docker

막힐 때

- `docker`를 인식하지 못함 → Docker Desktop을 설치·실행했는지 확인하고 PowerShell을 새로 엽니다.
- `Cannot connect to the Docker daemon` → Docker Desktop이 꺼져 있습니다. 켜고 'Engine running'이 될 때까지 기다립니다.
- `no configuration file provided` 또는 Dockerfile을 못 찾음 → `Dockerfile`, `compose.yaml`이 있는 폴더에서 실행했는지 확인합니다. 메모장으로 만들었다면 `Dockerfile.txt`처럼 `.txt`가 붙지 않았는지 봅니다.
- 빌드가 오류·시간 초과(`apt` · `npm` · `toomanyrequests`) → 학교 네트워크 차단이나 Docker Hub 한도일 수 있습니다. 개인 핫스팟으로 다시 하거나 강사의 이미지 파일로 `docker load`를 씁니다.

해설 · 발표자 노트

소요 시간 1분

이 슬라이드는 4번에서 설치가 막힌 Windows 참가자만 따라 한다. 나머지는 8번으로 간다. 해당자는 4번과 8번 시간에 병행해서 하고, 진행자가 안내하는 시간은 1분이다. 키트 폴더는 강사가 미리 나눠 준다. 강조할 점: 빌드는 처음 한 번만 하면 되고, 이후에는 7번의 `docker compose run`만 쓴다. Dockerfile이 하는 일을 한 줄씩만 말한다: Node.js 22가 든 리눅스에서 시작 → Git·gh 설치 → npm으로 Claude Code와 Firebase CLI 설치 → 일반 사용자(root가 아님)로 실행. 이 파일들을 직접 쓰는 법은 다루지 않는다(복사해서 쓴다). 이미지는 설치 완료본이고 컨테이너는 그 이미지를 실행한 것이다. 이 Mac에서는 빌드가 약 1분 걸렸고 이미지가 1.8GB였다. 학교

네트워크나 Windows에서는 더 걸릴 수 있으므로 시간이 길어지면 앞 슬라이드 설명을 이어가며 기다린다. 이미지 파일을 쓰는 경우: Apple Silicon Mac에서 만든 이미지 파일은 Intel/AMD Windows에서 실행되지 않으므로, 강사는 Windows x64 PC에서 빌드해 `docker save claude-workshop -o claude-workshop.tar`로 만들거나 `--platform linux/amd64`로 빌드한다(약 487MB). 검증 범위: macOS(Apple Silicon)의 Docker Desktop 29.4에서 `docker compose build`, 실행, `docker save` / `docker load`를 확인했다. Windows 실기 빌드는 확인하지 못했다 확인 필요. 원본 대비 변경: 원본에 없는 내용. (1분)

도식 설명 가로 흐름 3칸. `Dockerfile`(설계도: Node 22 + Git·gh + Claude Code·Firebase CLI) → `docker compose build`(한 번 실행) → 이미지 `claude-workshop`(설치 완료본). 아래에 작은 글씨로 "이미지 = 설치 완료본, 컨테이너 = 그 이미지를 실행한 것(7번)".

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- Windows 리허설

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

슬라이드 7 / 71

실습 1분 확인 필요

7 Docker로 Claude Code 시작하기 (해당자만)

① Docker Desktop이 켜져 있는지 확인하고, 키트 폴더에서 PowerShell을 엽니다.
② 아래 명령으로 Claude Code를 시작합니다. 6번에서 만든 이미지로 새 컨테이너가 열립니다.
③ 터미널에 나온 로그인 주소를 복사해 브라우저에서 열고, 화면에 나온 코드를 터미널에 붙여넣습니다.
④ 종료는 `/exit`. 로그인과 `work` 폴더의 파일은 남습니다.

프롬프트 · 파일

(PowerShell에 입력)

```
cd C:\claude-docker
docker compose run --rm claude
```

예상 결과

Claude Code 시작 화면이 나오고, 로그인하면 8번과 같은 첫 문장을 그대로 붙여넣을 수 있다. 컨테이너를 닫아도 로그인과 `work` 폴더의 파일은 남는다. 다음부터는 이 명령만 쓰면 된다(빌드는 다시 하지 않는다). 막히면 설명서: dshs-gemini.github.io/harness-workshop/docker

막힐 때

- `Cannot connect to the Docker daemon` → Docker Desktop이 꺼져 있습니다. 켜고 'Engine running'이 될 때까지 기다립니다.
- 이미지를 찾을 수 없다고 나오거나 빌드가 다시 시작됨 → 6번 빌드(또는 `docker load`)가 끝났는지 확인합니다.
- 로그인 브라우저가 자동으로 안 뜸 → 정상입니다. 터미널의 주소를 직접 복사해 열고, 나온 코드를 터미널에 붙여넣습니다.
- `docker compose down -v`는 쓰지 않습니다. 로그인 정보까지 지워집니다.

해설 · 발표자 노트

소요 시간 1분

4번에서 설치가 막힌 Windows 참가자만 따라 한다. 나머지는 8번으로 간다. 해당자는 8번 시간에 병행하며, 진행자가 안내하는 시간은 1분이다. 작업 폴더는 `C:\claude-docker\work`이고 컨테이너 안에서는 `/workspace`다. 이 폴더의 파일만 내 PC와 공유되므로 수업 자료는 여기에 만든다. `--rm`은 닫을 때 컨테이너를 지운다는 뜻이며, 로그인 정보(`claude-config` 볼륨)와 `work` 폴더는 따로 보관되어 남는다. 로그인은 컨테이너에 브라우저가 없어서 주소를 직접 복사해 열어야 한다. 브라우저가 완료되지 않으면 화면에 코드가 나오며, 그 코드를 터미널의 `Paste code here if prompted`에 붙여넣는다(공식 devcontainer 문서 안내). 이 경로로 시작한 참가자는 이후 8번 첫 대화를 같은 화면에서 한다. Firebase CLI·Git·gh는 이미지에 들어 있어 M4 25번·M6 50번에서 따로 설치하지 않는다. `firebase login`은 브라우저가 없으므로 `firebase login --no-localhost`를 쓴다(M6 50번 막힐 때 참고). 검증 범위: macOS(Apple Silicon)의 Docker Desktop 29.4에서 실행을 확인했다(`claude --version` 2.1.312, Git·gh·Firebase CLI 동작, `work` 폴더 공유, 로그인 폴더 유지). Windows 실기와 계정 로그인 완료, 하네스 플러그인 설치, Firebase 로그인 완료는 확인하지 못했다 확인 필요. 원본 대비 변경: 원본에 없는 내용. (1분)

도식 설명 터미널 2단 화면. 위: `cd C:\claude-docker` → `docker compose run --rm claude`. 아래: Claude Code 입력 줄. 오른쪽에 폴더 연결 그림: `C:\claude-docker\work` ⇄ `/workspace`.

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- Windows 리허설

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

8

슬라이드 8 / 71

실습 3분

실습 폴더에서 첫 대화

실습 폴더로 이동(`cd`)하고 `claude`를 실행합니다.
처음이면 브라우저에서 로그인하고 터미널로 돌아옵니다.
첫 문장을 그대로 붙여넣고 Enter.

프롬프트 · 파일

(터미널에서 먼저 아래 두 줄 실행, 그 뒤 Claude Code 화면에 붙여넣기)

```
cd Desktop/claude-training
claude
```

```
너는 지금 이 폴더에서 무엇을 할 수 있어? 교사인 내가 수업 자료를 만드는 걸 어떻게 도와줄 수 있는지 3가지만 예를 들어줘
```

예상 결과

Claude가 "이 폴더는 비어 있다", "파일을 만들고 실행할 수 있다"를 바탕으로 수업 자료 제작 예시 3가지를 구체적으로 제안한다. 이것이 이번 모듈의 산출물(실습 폴더에서 실행한 첫 대화 화면)이다. 응답 문구는 사람마다 다르다.

막힐 때

- 로그인 브라우저 창이 안 뜸 → 방화벽·보안 프로그램이 원인인 경우가 많습니다. 개인 핫스팟으로 잠시 시도합니다.
- `cd`가 안 됨(폴더 없음) → 실습 폴더 `claude-training`을 바탕화면에 먼저 만듭니다. 경로가 다르면 폴더 위치를 확인합니다.
- Docker 경로(5~7번)로 시작한 경우 → `cd`는 건너뜁니다. 7번에서 연 Claude Code 화면에 아래 첫 문장을 그대로 붙여넣습니다. 작업 폴더는 `C:\claude-docker\work`입니다.

해설 · 발표자 노트

소요 시간 3분

`cd`는 폴더를 이동하는 명령이다. 첫 실행 때 브라우저에서 Claude 계정으로 로그인하고 터미널로 돌아오면 현재 폴더 이름과 함께 대화 준비 화면이 나온다. 아직 파일이 없는 빈 폴더라서 "무엇을 할 수 있는지"를 묻는 것이 좋은 첫 질문이다. 프롬프트는 원본 그대로. 대화 중 화면을 잘 보도록 안내: 사람은 결과를 검토하고 승인하는 쪽이다. 폴더 이름 `claude-training`은 사전에 만들어 둔 것(원본 intro). 원본 대비 변경: 없음(4~5단계 압축). (3분)

도식 설명 터미널 2단 화면. 위: `cd` + `claude` 실행. 아래: `>` 입력 줄에 첫 문장 입력. 오른쪽에 "빈 폴더 `claude-training`"을 폴더 아이콘으로 표시.

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

9

슬라이드 9 / 71

막히면 이렇게

정리 1분

로그인 창이 안 뜨면 → 핫스팟 · `command not found` → 터미널 재시작
Windows 설치 오류가 계속되면 → Docker 경로(5~7번), 설명서: dshs-gemini.github.io/harness-workshop/docker
대화를 끝낼 때는 `/exit` (또는 `Ctrl+D` 두 번)

해설 · 발표자 노트

소요 시간 1분

이 슬라이드는 실습 시간으로 센다. 막힌 참가자를 돌아보며 확인한다. 설치·로그인이 끝내 안 되는 분은 짝 화면을 함께 보거나 강사 화면을 따라 본다. 다음 모듈 예고: 잘 시키는 법(구체적 프롬프트, 컨텍스트, 플랜 모드). (1분)

도식 설명 3칸 표. 증상 / 해결 / 종료법. Windows 설치 오류 행의 해결 칸은 "Docker 경로(5~7번)". 아래 작은 상자에 짝 화면 함께 보기, 강사 화면 따라 보기 안내.

MODULE M2

모듈 M2

잘 시키는 법: 프롬프트 · 컨텍스트 · 플랜 모드

학습 목표

> 플랜 모드로 계획을 먼저 확인하고 구체적인 프롬프트로 파일을 만들고 고칠 수 있다.

시간	**12분**
슬라이드	**10-14 (5장)**
실습 산출물	`lesson-01.md`

이 모듈의 슬라이드

번호	제목	유형	시간	쪽
10	모호한 요청 vs 구체적인 요청	개념	2분	·
11	컨텍스트와 플랜 모드	개념	2분	·
12	실습: 플랜 모드로 수업 개요 만들기	실습	4분	·
13	실습: 파일 확인하고 이어서 고치기	실습	3분	·
14	방금 무슨 일이 있었나	정리	1분	·

10 모호한 요청 vs 구체적인 요청

슬라이드 10 / 71

개념 2분

- Claude Code에게도 작업 지시는 구체적으로 한다.
- 문장에 **대상 · 범위 · 형식 · 기준** 네 가지를 담는다.
- 다시 요청할 일이 줄어든다.

도식 · 단계

- 나쁜 예(모호한 요청): "퀴즈 좀 만들어줘"
- 좋은 예(구체적인 요청): "고1 정보 수업 '변수와 자료형' 단원 복습용으로, 객관식 5문항짜리 퀴즈를 만들어줘. 각 문항에 정답과 한 줄 해설을 붙이고, 난이도는 쉬움 3개·보통 2개로 섞어줘"
- 좋은 예 문장 아래에 네 요소를 색으로 표시: 대상(고1 정보 수업), 범위('변수와 자료형' 단원 복습용), 형식(객관식 5문항, 정답과 한 줄 해설), 기준(쉬움 3개·보통 2개).

해설 · 발표자 노트

소요 시간 2분

(발표자 대본) "같은 퀴즈 요청인데 오른쪽 카드는 누가, 무엇을, 어떤 모양으로, 어떤 기준으로 만들지 다 들어 있습니다. 네 가지를 빼먹으면 Claude Code가 추측해서 만들고, 그러면 다시 요청해야 합니다." 짝에게 왼쪽 카드를 고쳐 말해 보게 하면 좋다(30초, 선택). 담당 과목에 맞게 바꿔 말해도 된다고 안내한다. (2분)

도식 설명 카드 2장 비교(원본 `.grid2` 의 `card bad` / `card good`).

---

11 컨텍스트와 플랜 모드

슬라이드 11 / 71

개념 2분

- 컨텍스트 = 모델 입력으로 유지되는 대화·파일 내용(컨텍스트 윈도우). 대화가 길어지면 관련 없는 내용이 쌓여 답이 흐려질 수 있다.
- 플랜 모드 = 파일 수정 없이 읽기 전용으로 계획만 세우고, 승인 후 실행한다. `Shift` + `Tab` 으로 켠다.
- 하단 상태 표시줄에서 지금 모드를 확인한다.

도식 · 단계

- | 명령 | 동작 | 언제 쓰나 |

`/clear`	**대화 기록을 초기화합니다 (컨텍스트 비우기)**	**완전히 다른 작업을 새로 시작할 때**
`/compact`	지금까지의 대화를 요약해서 압축합니다 (요약본만 남기기)	같은 작업을 계속하되 맥락은 유지하고 싶을 때

- 요청 → 계획만 제시(파일 안 고침) → 사람이 승인 → 실제 작업. 아래에 화면 하단 상태 표시줄 위치 표시.

해설 · 발표자 노트

소요 시간 2분

(발표자 대본) "Claude Code는 이전 요청과 읽은 파일을 컨텍스트로 기억합니다. 컨텍스트가 길어지면 `/clear` 로 대화 기록을 초기화하고, 같은 작업을 이어 가되 요약본만 남기고 싶으면 `/compact` 입니다. 파일을 여러 개 만들거나 고치는, 되돌리기 번거로운 일은 먼저 플랜 모드로 계획을 보고 승인합니다." 덧붙임: `Shift` + `Tab` 은 플랜 모드뿐 아니라 이후 M3에서 다룰 권한 모드(도구 실행 승인 정책)를 순환 전환하는 단축키이기도 하다. 하단 상태 표시줄에서 현재 모드를 확인한다. 모드 표시는 하단에 `⏸ plan mode on` 같은 문구로 보인다(M3 21 표 참고). (2분)

도식 설명 왼쪽에 표, 오른쪽에 플랜 모드 흐름.

---

12 슬라이드 12 / 71

실습 4분

실습: 플랜 모드로 수업 개요 만들기

- [ ] **1.** `/clear`로 대화 기록을 초기화한다.
- [ ] **2.** `Shift`+`Tab`을 하단에 `⏸ plan mode on`이 보일 때까지 누른다(Auto로 시작했다면 세 번). 또는 프롬프트 앞에 `/plan`을 붙인다.
- [ ] **3.** 아래 프롬프트를 붙여 넣고, 계획을 읽은 뒤 승인 화면에서 **"Yes, manually approve edits"**를 고른다.

프롬프트 · 파일

```
고1 정보과학 '조건문과 반복문' 1차시(50분) 수업 개요를 markdown 파일로 만들어줘.
도입(5분)-전개(35분)-정리(10분) 구조로 나누고, 전개 단계에는 학생이 직접 코드를
고쳐보는 미니 실습을 하나 포함해줘. 파일 이름은 lesson-01.md로 해줘
```

예상 결과

터미널 하단에 `⏸ plan mode on`이 보이고, 파일을 만들기 전 계획이 먼저 나온다.

```
계획: lesson-01.md 파일을 새로 만들겠습니다. 구성은 다음과 같습니다…
1. Yes, and use auto mode
2. Yes, manually approve edits   <- 연수에서는 이것을 고른다
3. No, keep planning
```

계획을 읽고 "Yes, manually approve edits"를 고르면 폴더에 `lesson-01.md`가 생긴다. "manually approve edits"를 고르면 세션이 Manual 모드가 되므로 파일을 만들 때 허용을 한 번 더 묻는다. 그때 허용한다. 1번을 고르면 세션이 Auto 모드로 바뀌므로 연수에서는 고르지 않는다. 3번은 계획을 더 고치게 하고 싶을 때 쓴다.

막힐 때

- `Shift`+`Tab`을 눌러도 plan 표시가 안 나옴 → 순환 순서는 Manual → Accept edits → Plan이다(Auto로 시작했다면 첫 번째 누름에 Manual로 바뀐다). 하단 표시줄에서 `⏸ plan mode on`이 보일 때까지 계속 누르거나, 프롬프트 앞에 `/plan`을 붙인다.
- 승인 화면의 선택지가 위와 조금 다름 → 화면에서 "manually approve edits"에 해당하는 것을 고른다. 모르겠으면 손을 든다.

해설 · 발표자 노트

소요 시간 4분

(발표자 대본) "먼저 대화 기록을 초기화합니다. 이전 실습 세션이 있다면 `/clear`. 그다음 `Shift`+`Tab`으로 플랜 모드. 이 프롬프트를 보면 대상(고1 정보과학), 범위(1차시 50분), 형식(markdown, 도입-전개-정리, 파일 이름), 기준(미니 실습 하나)이 다 있죠. 계획이 나오면 읽어 보고 두 번째 'manually approve edits'를 고르세요. 첫 번째를 고르면 Auto 모드로 바뀌어 이후 확인이 줄어듭니다. 파일이 만들어지기 전에 사람이 확인한다는 점이 핵심입니다." 용어: markdown = 제목·목록을 기호(#, -)로 적는 간단한 글 형식. 담당 과목에 맞게 과목·단원을 바꿔도 된다고 안내한다. 승인 후 `lesson-01.md`가 생성될 때까지 기다린다. (4분)

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

13

슬라이드 13 / 71

실습: 파일 확인하고 이어서 고치기

실습 3분

☐ **1.** 평소 쓰던 편집기(메모장, VS Code 등)로 `lesson-01.md`를 열어 확인한다.

☐ **2.** 마음에 안 드는 부분은 새로 만들지 말고 이어서 수정을 요청한다.

☐ **3.** 무엇을 바꿀지만 말하면 된다(이전 내용이 컨텍스트에 남아 있다).

프롬프트 · 파일

```
전개 단계 미니 실습을 더 쉽게 바꿔줘. for문 대신 if문만 쓰는 문제로 바꿔줘
```

예상 결과

Claude Code가 방금 만든 `lesson-01.md`의 전개 단계 미니 실습을 if문만 쓰는 쉬운 문제로 고친다. 다시 파일을 열면 바뀐 내용이 보인다. (과목을 바꿔 실습한 경우 그에 맞는 쉬운 수정 요청으로 바꾼다.)

막힐 때

- `lesson-01.md`를 찾을 수 없음 → 터미널에서 실습 폴더(`claude-training`)를 연 상태인지 확인하고, 폴더 안에서 파일 목록을 열어 본다.
- 파일 수정 허용을 물음 → 허용한다. 계획 화면이 다시 나오면 플랜 모드가 켜져 있는 것이니 `Shift`+`Tab`으로 Manual로 바꾼다.

해설 · 발표자 노트

소요 시간 3분

(발표자 대본) "터미널이 아니라 평소 쓰는 편집기로 파일을 열어 보세요. 사람이 눈으로 확인하는 단계입니다. 그리고 고치고 싶은 부분이 있으면 처음부터 다시 만들지 말고 '더 쉽게 바꿔줘'처럼 이어서 말하세요. Claude Code는 방금 만든 파일을 기억하고 있습니다." 먼저 끝난 참가자는 다른 요소(시간 배분, 정리 단계 질문 추가 등)를 이어서 고쳐 보게 한다. (3분)

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

---

14 슬라이드 14 / 71

방금 무슨 일이 있었나

정리 1분

- 구체적인 프롬프트 → 플랜 모드 → 컨텍스트 유지, 이 세 가지가 방금 실습에 모두 들어 있었다.
- 앞으로 모든 실습이 이 세 가지 위에서 이루어진다.

도식 · 단계

- ① 구체적인 프롬프트(대상·시간·구조·형식 지정) → ② 플랜 모드(파일을 만들기 전 계획 확인) → ③ 컨텍스트 유지(다시 설명하지 않고 "더 쉽게 바꿔줘"만으로 이어서 수정)
- 하단에 `lesson-01.md` 파일 아이콘 + 라벨 "나의 첫 산출물".

해설 · 발표자 노트

소요 시간 1분

(발표자 대본) "짧은 실습이었지만 세 가지가 다 들어 있었습니다. 구체적으로 말했고, 계획을 먼저 확인했고, 설명을 반복하지 않고 이어서 고쳤죠. 다음 모듈부터는 이 말하기 방식 위에 규칙(CLAUDE.md)과 스킬을 얹습니다." 이 슬라이드는 실습 시간으로 센다. (1분)

도식 설명 `.flow` 3단 도식.

MODULE M3

모듈 M3

CLAUDE.md · 스킬 · 권한

학습 목표

> CLAUDE.md로 반복 규칙을 남기고, 나만의 스킬(/quiz)을 만들어 재사용할 수 있다.

시간	**19분**
슬라이드	**15-22 (8장)**
실습 산출물	`CLAUDE.md` + `.claude/skills/quiz/SKILL.md`

이 모듈의 슬라이드

번호	제목	유형	시간	쪽
15	CLAUDE.md = 세션 시작 시 자동 로드되는 프로젝트 지침	개념	2분	·
16	실습: /init 으로 CLAUDE.md 만들고 규칙 5개 추가	실습	4분	·
17	실습: 규칙이 지켜지는지 확인하기	실습	2분	·
18	슬래시 커맨드와 스킬 = 저장된 절차서	개념	1분	·
19	실습: /quiz 스킬 만들기	실습	4분	·
20	실습: /quiz 로 주제만 바꿔 재사용	실습	2분	·
21	권한 모드: 도구 실행 승인 정책	개념	2분	·
22	점검: Track 1 체크포인트 6문장	점검	2분	·

15 슬라이드 15 / 71

CLAUDE.md = 세션 시작 시 자동 로드되는 프로젝트 지침

개념 2분

- CLAUDE.md는 폴더 맨 위에 두는 텍스트 파일. 세션을 시작할 때마다 컨텍스트에 자동 로드되는 프로젝트 지침 파일
- 적는 것 4가지: 산출물 형식 · 대상 독자와 어조 · 반복되는 제약 · 절대 하지 말 것
- 한계: "부탁"이지 "강제"가 아니다

해설 · 발표자 노트

소요 시간 2분

"1.3에서 컨텍스트는 대화를 새로 시작하면 초기화된다고 했습니다. 매번 '한국어로, HTML 한 파일로'를 다시 말하기 싫다면 CLAUDE.md에 적어 두면 세션 시작 때마다 자동으로 로드됩니다." 예시 4개는 원본 그대로: 형식("학생 배포용 자료는 항상 자급형 HTML 한 파일"), 독자·어조("고1 대상, 존댓말, 어려운 용어는 처음 나올 때 한 줄 설명"), 반복 제약("수식은 KaTeX, `\` 이스케이프 주의"), 금지("학생 이름·학번이 든 예시 데이터 만들지 않기"). 그냥 텍스트 파일이라 메모장으로 열어 고쳐도 되고 Claude에게 고쳐 달라고 해도 된다. 한계 설명은 짧게만: "CLAUDE.md는 컨텍스트에 들어가는 지침이라 모델이 가끔 따르지 못할 수 있습니다. 그래서 결과는 사람이 직접 확인합니다." (2분)

도식 설명 `CLAUDE.md` 파일 그림 한 장. 4개 카드(형식 / 독자·어조 / 반복 제약 / 금지)가 파일 안의 항목으로 들어 있고, 오른쪽에 "대화를 새로 시작해도 다시 읽힘" 화살표. 맨 아래 경고 띠: "산문으로 적은 부탁 → 가끔 놓칠 수 있음".

16 슬라이드 16 / 71

실습: /init 으로 CLAUDE.md 만들고 규칙 5개 추가

실습 4분

- 1단계: `/init` 입력. 폴더를 훑어 기본 뼈대 CLAUDE.md를 만든다
- 2단계: 아래 프롬프트를 붙여 넣어 우리 규칙 5개를 추가
- 직접 타자 치지 않는다. 붙여 넣기만

프롬프트 · 파일

1단계

```
/init
```

2단계

```
CLAUDE.md에 다음 규칙을 추가해줘:
  1. 이 폴더는 고등학교 정보과 수업 자료 제작용이다
  2. 학생 배포용 산출물은 항상 외부 파일 참조가 없는 자급형 HTML 한 파일로 만든다
  3. 본문은 한국어, 코드와 전문 용어는 원어를 유지한다
  4. 학생 이름·학번 등 개인정보가 들어간 예시 데이터는 절대 만들지 않는다
  5. 코드 예제는 별도 요청이 없으면 파이선으로 작성한다
```

예상 결과

폴더에 `CLAUDE.md`가 생기고, 열어 보면 뼈대 아래에 규칙 5개가 들어 있다. 파일 수정 승인을 물으면 허용한다.

막힐 때

- `/init`이 반응이 없으면 `claude`를 실습 폴더(`claude-training`)에서 실행했는지 확인. 폴더가 비어 있어도 뼈대는 만들어진다.
- 다른 과목이면 1번 규칙의 "정보과"를 내 과목으로 바꿔서 붙여 넣는다.

해설 · 발표자 노트

소요 시간 4분

순서대로 진행. 먼저 `/init`(기본 뼈대 자동 생성), 다음에 규칙 추가. 학교급·과목은 바꿔도 된다고 안내. 완성되면 편집기로 CLAUDE.md를 열어 보게 한다. 막힌 참가자는 짝 화면을 함께 본다. (4분)

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

17

슬라이드 17 / 71

실습: 규칙이 지켜지는지 확인하기

실습 2분 확인 필요

- 형식을 말하지 않고 요청한다
- 자급형 HTML · 한국어 · 파이선 예제로 나오면 성공
- 규칙은 폴더마다, 전역 규칙은 `~/.claude/CLAUDE.md`

프롬프트 · 파일

```
조건문 개념을 설명하는 학생용 학습 자료를 하나 만들어줘
```

예상 결과

형식을 다시 설명하지 않았는데도, CLAUDE.md 규칙에 따라 자급형 HTML 파일로, 한국어 본문에 파이선 코드 예제를 넣어 생성한다. (생성 내용은 실행마다 다르다.)

막힐 때

- 규칙이 반영되지 않은 것 같으면 `/exit` 후 `claude`를 다시 실행(세션 시작 때 CLAUDE.md를 읽는다)하고 같은 요청을 한다 확인 필요.
- 파일이 HTML이 아니면 "CLAUDE.md의 규칙 2번을 지켜서 다시 만들어줘"라고 말한다.

해설 · 발표자 노트

소요 시간 2분

목적은 자료 품질이 아니라 "규칙이 말하지 않아도 적용되는지" 확인하는 것. 이 슬라이드가 CLAUDE.md의 효과를 눈으로 보는 순간이다. 여러 폴더를 오가는 교사 팁: 과목별·업무별 폴더마다 다른 CLAUDE.md를 둘 수 있고, 모든 프로젝트에 공통인 규칙(예: "항상 한국어로 답한다")은 홈 폴더의 `~/.claude/CLAUDE.md`(전역)에 적으면 어느 폴더에서 켜도 적용된다. 개인정보 금지 규칙(4번)은 이후 모든 실습에서 계속 작동한다는 점을 강조. (2분)

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 같은 세션에서 수정한 CLAUDE.md가 즉시 반영되는지

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

18

슬라이드 18 / 71

슬래시 커맨드와 스킬 = 저장된 절차서

개념 1분

- `/init` `/clear` 같은 슬래시 커맨드는 미리 만들어진 명령
- 스킬 = 내가 자주 시키는 요청을 저장해 둔 업무 매뉴얼(절차서)
- 명령 이름 = `name` 값(없으면 폴더 이름). 예: `.claude/skills/quiz/SKILL.md` → `/quiz`

도식 · 단계

명령	기능
`/help`	사용 가능한 명령 목록
`/clear`	대화 기록 초기화
`/compact`	대화를 요약해 압축
`/init`	CLAUDE.md 뼈대 생성
`/permissions`	권한(허용 범위) 확인·수정 (슬라이드 21)
`/agents`	서브에이전트 관리 안내(예전 버전은 목록·생성 화면)
`/skills`	설치된 스킬 목록
`/exit`	Claude Code 종료

해설 · 발표자 노트

소요 시간 1분

표는 읽지 않고 훑는다. 앞쪽 몇 개는 이미 써 보았다. `/permissions`·`/agents`는 "나중에 쓴다"만 말하고 넘어간다(`/agents`는 요즘 버전에서 목록 대신 "서브에이전트는 Claude에게 만들어 달라고 하라"는 안내를 보여 준다). 이름 규칙: `name`을 쓰면 그 이름이 `/` 메뉴의 명령 이름이 된다. 폴더 이름으로도 호출할 수 있지만 명령 이름은 `name` 값이 우선이고, `name`이 없으면 폴더 이름이 명령 이름이다. 핵심 한 줄: "내가 반복해서 시키는 요청 자체를 나만의 슬래시 커맨드로 저장한 것이 스킬." 스킬은 평소엔 description 한 줄만 기억하고(가벼움), 필요한 순간에만 본문 전체를 읽는다. 이것이 점진적 공개(progressive disclosure)이며 대화창이 불필요하게 무거워지지 않게 하는 장치다. 다음 슬라이드에서 직접 만든다고 예고. (1분)

도식 설명 왼쪽에 표 8행(아래), 오른쪽에 폴더 도식 `.claude/skills/quiz/SKILL.md` → `/quiz` 화살표. SKILL.md 상단에 "description(한 줄, 늘 기억)"과 "본문(필요할 때만 읽음)" 두 칸.

19 슬라이드 19 / 71

실습: /quiz 스킬 만들기

실습 4분

- 파일 경로와 형식을 몰라도 된다. Claude Code에게 부탁한다
- 만들어진 SKILL.md를 폴더에서 직접 열어 확인
- `disable-model-invocation: true` = 내가 `/quiz`라고 부를 때만 실행

프롬프트 · 파일

```
.claude/skills/quiz/ 에 SKILL.md를 만들어줘.
  이름은 quiz, 설명은 "주제를 입력하면 객관식 퀴즈를 생성한다. 사용자가
  /quiz라고 명시적으로 호출했을 때만 실행한다"로 하고,
  $ARGUMENTS로 받은 주제에 대해 객관식 5문항(정답+한 줄 해설 포함)을
  만들도록 본문을 작성해줘. 난이도는 쉬움 2 · 보통 2 · 어려움 1로 섞어줘
```

예상 결과

아래와 비슷한 파일이 생성된다(문구는 실행마다 조금 다를 수 있다).

`.claude/skills/quiz/SKILL.md`

```
---
name: quiz
description: 주제를 입력하면 객관식 퀴즈를 생성한다. 사용자가 /quiz라고
  명시적으로 호출했을 때만 실행한다.
disable-model-invocation: true
argument-hint: [주제]
---

## 지시사항

$ARGUMENTS 주제에 대한 객관식 퀴즈 5문항을 만든다.

- 난이도 구성: 쉬움 2문항, 보통 2문항, 어려움 1문항
- 각 문항은 4지선다, 정답 표시, 한 줄 해설 포함
- 학생이 헷갈릴 만한 오답 보기를 최소 1개 이상 포함
- 출력은 markdown 표 형식으로 정리
```

보조 설명: `disable-model-invocation: true`가 있으면 Claude가 눈치껏 알아서 퀴즈를 만들지 않고, 사람이 `/quiz`라고 직접 호출했을 때만 실행된다. 배포·삭제처럼 자동 실행되면 곤란한 명령에 특히 유용.

막힐 때

- 폴더가 `.claude/skills/quiz/`가 아니거나 파일 이름이 `SKILL.md`가 아니면 `/quiz`가 인식되지 않는다. 폴더를 열어 이름을 확인하고, 틀리면 "SKILL.md 위치를 .claude/skills/quiz/로 옮겨줘"라고 말한다.
- 파일이 만들어졌는데 `/quiz`가 목록에 안 보이면 `/skills`로 확인하고, 안 보이면 `/reload-skills`를 입력한다(세션을 시작할 때 `.claude/skills/` 폴더가 없었다면 새 스킬이 자동으로 잡히지 않는다).

해설 · 발표자 노트

소요 시간 4분

프롬프트는 원본 그대로. 프론트매터 키(`name`, `description`, `disable-model-invocation`, `argument-hint`)를 file-box에서 하나씩 짚어 준다: name = 명령 이름, description = Claude가 평소에 기억하는 한 줄, `argument-hint` = 입력창에 보이는 안내, `$ARGUMENTS` = 내가 입력한 주제가 들어갈 자리. 참가자가 손으로 파일을 쓰게 하지 않는다. 용어: 프론트매터(frontmatter) = 파일 맨 위 `---` 사이의 설정 칸. (4분)

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

---

20 슬라이드 20 / 71

실습: /quiz 로 주제만 바꿔 재사용

실습 2분

- 주제만 말하면 5문항 퀴즈 표가 나온다
- 주제를 바꿔 계속 재사용
- 매번 형식을 설명하지 않아도 된다 = 스킬의 핵심

프롬프트 · 파일

```
/quiz 재귀함수
```

이어서 주제를 바꿔 한 번 더 (내 과목 주제로 바꿔도 됨)

```
/quiz 반복문
```

예상 결과

재귀함수를 주제로 한 5문항짜리 객관식 퀴즈 표가 생성된다(쉬움 2·보통 2·어려움 1, 정답과 한 줄 해설 포함). 문항 내용은 실행마다 다르다.

> 막힐 때
> - "Unknown command"류 메시지가 나오면 `/skills`로 quiz가 있는지 확인(슬라이드 19 막힐 때 참고).
> - 표가 아니라 줄글로 나오면 "SKILL.md의 출력 형식대로 다시 해줘"라고 말한다.

해설 · 발표자 노트

소요 시간 2분

짝에게 자기 과목 주제로 `/quiz`를 돌려 보여 주게 한다(예: `/quiz 리스트와 딕셔너리`). 스킬이 `.claude/skills/` 아래 폴더의 `SKILL.md` 파일이라는 점은 M4에서 하네스가 만들어 주는 파일과 같은 형태이므로 눈여겨보게 한다. 이 단계에서 만든 `.claude/skills/quiz/SKILL.md`와 `CLAUDE.md`가 M3 실습 산출물이다. (2분)

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

21

슬라이드 21 / 71

개념 2분

권한 모드: 도구 실행 승인 정책

- 권한 모드 4가지: 일반(Manual) · 편집 자동 승인(Accept edits) · 계획(Plan) · Auto
- 최신 버전은 Auto 모드로 시작한다. 처음 배울 때는 `Shift+Tab` 한 번으로 Manual로 바꾼다. 하단 상태 표시줄에서 확인
- 모드 = 세션 전체의 기본 승인 수준. `permissions` 규칙(`allow` · `ask` · `deny`, `/permissions`) = 도구·명령 단위 예외. 평가 순서는 deny → ask → allow, deny는 모든 모드에서 적용되고 allow는 해당 호출의 승인 질문을 생략한다
- 경고: 개인정보 폴더·실제 배포에는 완전 자동 승인 금지

도식 · 단계

모드	동작	하단 표시
일반(Manual)	파일 수정·명령 실행 전마다 승인을 구한다. 처음 배울 때 가장 안전	`⏸ manual mode on`
편집 자동 승인(Accept edits)	파일 수정은 묻지 않고 진행, 명령 실행 등은 승인을 구한다	`▸▸ accept edits on`
계획(Plan)	어떤 변경도 실행하지 않고 계획만 제시 (M2에서 사용)	`⏸ plan mode on`
Auto	분류 모델이 대신 검토해 위험도 낮은 작업은 자동 진행	`▸▸ auto mode on`

경고 띠: "학생 성적·개인정보가 든 폴더, 실제 서비스 배포 → 완전 자동 승인 금지. 사람이 눈으로 확인하는 단계를 남긴다"

해설 · 발표자 노트

소요 시간 2분

권한 모드는 Claude Code가 도구(파일 수정, 명령 실행)를 실행할 때 사용자 승인을 어디까지 요구할지 정하는 정책이다. 원본의 3모드 표를 공식 권한 모드 문서에 맞춰 4모드로 바꿨다(원본 대비 변경). Shift+Tab 순서: Auto에서 한 번 누르면 Manual, 이어서 Accept edits, Plan 순이다. 시작 모드가 Auto인 것은 Claude Code v2.1.283 이상 기준이므로 화면 하단 표시가 다르면 화면 기준으로 안내한다. 개인정보가 든 폴더에서는 시작하자마자 `Shift+Tab`으로 Manual로 바꾸게 한다(완전 자동 승인 금지 원칙). 모드와 permissions 규칙의 관계: 모드는 규칙에 걸리지 않은 호출에 적용되는 기본 정책이고, 규칙은 그 위의 예외다. 규칙은 settings.json의 `permissions.allow/ask/deny`에 `Bash(npm run *)`, `Read(./.env)` 형식으로 쓰고 `/permissions`로 확인·추가·삭제한다. 평가는 deny → ask → allow 순이고 먼저 일치한 규칙이 결정한다. 넓은 deny는 구체적인 allow보다 우선하며 allow로 deny의 예외를 만들 수 없다. deny는 bypassPermissions를 포함한 모든 모드에서 적용되고, default(Manual) 모드에서 allow는 해당 도구의 승인 질문을 생략한다. 예: Manual 모드 + `Bash(npm test)` allow + `Read(./.env)` deny. 적용 범위는 managed > 명령행 > 로컬(.claude/settings.local.json) > 프로젝트(.claude/settings.json) > 사용자(~/.claude/settings.json) 순이고 규칙 배열은 합쳐지되 deny가 항상 우선한다. 주의: Bash 규칙은 명령 문자열 매칭이라 보안 경계가 아니므로(예: 전체 경로나 `sh -c`로 우회 가능) 강한 차단이 필요하면 샌드박스를 함께 쓴다. 공식 문서(code.claude.com/docs/en/permissions) 기준. 교원 연수에서 특히 강조하는 원칙이므로 경고 띠는 소리 내어 읽는다. M6에서 Firebase 규칙 게시·배포 직전의 사람 확인으로 다시 만난다. (2분)

도식 설명 4행 표 + 아래 빨간 경고 띠.

22 슬라이드 22 / 71

점검: Track 1 체크포인트 6문장

점검 2분

- 6문장 중 "할 수 있다"에 손들기. 몇 개나 되나요?
- 1~2개 모자라도 괜찮다. 막힌 곳을 손들어 알려 주세요

도식 · 단계

1. Claude Code가 일반 채팅 AI와 다른 점(파일을 직접 읽고 쓰고 실행 결과를 스스로 확인한다는 점)을 한 문장으로 설명할 수 있다.
2. 내 컴퓨터에 Claude Code를 설치하고 로그인했다.
3. 플랜 모드(`Shift+Tab`)로 전환해 파일이 바뀌기 전에 계획을 먼저 확인해 봤다.
4. 내 폴더에 CLAUDE.md를 만들어 반복되는 규칙을 등록했다.
5. 나만의 슬래시 커맨드(`/quiz`)를 최소 한 번 만들어 보고 실행해 봤다.
6. 권한 모드가 왜 존재하는지, 언제 자동 승인을 피해야 하는지 설명할 수 있다.

해설 · 발표자 노트

소요 시간 2분

한 문장씩 읽고 손들기로 확인(1문장 약 15초). 2·4·5번에서 손이 적게 올라오면 해당 슬라이드(4-5, 13, 16)로 돌아가 짝 도움으로 5분 안에 보충. 이후 모듈이 CLAUDE.md와 `/quiz`를 계속 쓰므로 4·5번 미완료자는 짝 도움으로 보충한다. 원본 6문장은 그대로. 다음 모듈 연결: "여기까지는 Claude Code 혼자와 대화하는 법이었습니다. 다음 M4(슬라이드 23)에서는 하네스 플러그인을 설치해서, 말로 요청하면 역할이 나뉜 팀 파일을 만들어 주는 것을 써 봅니다." (2분)

도식 설명 체크박스 6줄(아래), 오른쪽에 손 든 인원 세는 칸.

점검 메모

MODULE M4

모듈 M4

하네스 입문

학습 목표

> 하네스(revfactory/harness) 플러그인을 설치하고, 말로 요청해 '만드는 쪽 + 검수하는 쪽' 팀을 만들어 실행할 수 있다.

시간	**21분**
슬라이드	**23-28 (6장)**
실습 산출물	`mini-harness/.claude/` **(퀴즈 만드는 에이전트 + 검수 에이전트 + 오케스트레이터 스킬)**

이 모듈의 슬라이드

번호	제목	유형	시간	쪽
23	하네스가 해 주는 일	개념	2분	·
24	실습: 플러그인 설치 전 준비 (새 폴더에서 시작)	실습	2분	·
25	실습: 플러그인 설치 (설치 통로 열고 설치하기)	실습	5분	·
26	실습: 말로 팀 만들기 (플랜 모드)	실습	5분	·
27	실습: 팀 실행하고 검수 결과 보기	실습	5분	·
28	정리: 하네스가 만든 것	정리	2분	·

23

슬라이드 23 / 71

하네스가 해 주는 일

개념 2분

- 혼자 맡기면: 자기 실수를 스스로 알아채기 어렵고, 모호한 요청을 스스로 해석해 끝까지 실행해 버린다
- 하네스 = 모델 바깥에서 도구·지침·검증·제어 흐름을 구성한 실행 환경
- `revfactory/harness` = 말로 요청하면 역할이 나뉜 팀 파일을 만들어 주는 플러그인
- 오늘 얻을 것: ① 플러그인 설치 ② 말로 팀 만들기 ③ 팀을 실행하고 검수 결과 보기

해설 · 발표자 노트

소요 시간 2분

여기까지는 Claude Code 혼자와 대화하는 법이었다고 전환한다. 혼자 맡기면 생기는 문제는 한 줄만 말한다. 코드를 쓴 바로 그 에이전트가 검토하면 놓친 부분을 또 놓치기 쉽고, "정리해줘" 같은 모호한 요청을 스스로 해석해 필요한 데이터까지 지우는 사고가 보고된 바 있다(원본 표현). 하네스 정의는 《하네스 엔지니어링 with 클로드 코드》(황민호, 한빛미디어) 인용이다: "모델의 가중치를 건드리지 않고, 모델 바깥에 권한·도구·검증·관측을 설계해 에이전트가 홀로 움직이게 만드는 환경." 하네스는 새 AI 모델이 아니라 기존 모델이 동작하는 조건(도구·지침·검증·제어 흐름)을 바꾸는 환경이다. 저자 주장 "모델이 아니라 하네스가 결과를 결정한다"와 평균 점수 49.5 → 79.3은 저자가 직접 한 소프트웨어 엔지니어링 과제 15개 실험이고 관련 저장소 README에도 제3자 재현 검증이 아직 없다고 적혀 있다. 방향성만 참고하고 개선폭은 하나의 사례로 말한다(단서 배지: 저자 자체 실험, 재현 검증 전). 이 연수는 하네스의 원리를 설계하는 시간이 아니라 `revfactory/harness`를 써서 팀 파일을 만들게 하고 고치는 시간이라고 분명히 안내한다. 구성 요소 이름(에이전트·스킬·오케스트레이터)은 슬라이드 28에서 한 줄씩만 정리한다. (2분)

도식 설명 가로 3칸 화살표 "말로 요청 → 하네스가 팀 파일 생성 → 팀 실행·검수". 칸 1에 말풍선 한 줄(예: "형성평가 문항을 만들고 검수하는 하네스를 구성해줘"), 칸 2에 `.claude/` 폴더 아이콘, 칸 3에 "만드는 쪽 / 검수하는 쪽" 두 상자. 아래에 작은 단서 배지 "수치 49.5 → 79.3: 저자 자체 실험, 재현 검증 전"은 노트에만 두고 슬라이드에는 넣지 않는다.

24

슬라이드 24 / 71

실습: 플러그인 설치 전 준비 (새 폴더에서 시작)

실습 2분

- 새 폴더 `mini-harness`에서 Claude Code를 연다
- 이 연수의 플러그인은 별도 설정(실험 플래그 등) 없이 바로 쓸 수 있다
- Git이 필요하다: 다음 슬라이드의 설치는 GitHub 저장소를 git으로 내려받는다(M1 4번에서 확인)
- 확인: 폴더 이름과 `claude` 실행 상태, `git --version`만 점검하고 다음 슬라이드로

프롬프트 · 파일

터미널에서:

```
git --version
mkdir mini-harness && cd mini-harness
claude
```

예상 결과

`git --version`에 버전이 나오고, Claude Code 입력창이 열리며, 현재 폴더가 `mini-harness`로 보인다.

막힐 때

- `git`을 찾을 수 없다고 나온다 → Windows는 Git for Windows(git-scm.com/downloads/win)를 설치하고 터미널을 새로 연다. 그 상태에서 `claude`를 다시 실행한다. git이 없으면 다음 슬라이드의 마켓플레이스 등록이 실패한다.
- `mkdir`가 안 되거나 폴더 위치를 모르겠다 → 지난 실습 폴더(`claude-training`) 바로 아래에 만들어도 된다. 이미 `mini-harness`가 있으면 그 폴더로 `cd`만 한다.

해설 · 발표자 노트

소요 시간 2분

`revfactory/harness` v2는 실험 플래그가 필요 없고, 플래그를 켜 두면 슬라이드 27의 quiz-writer/quiz-checker가 서브에이전트가 아니라 팀원(teammate)으로 실행되어 예시 화면과 달라질 수 있다. 그래서 이 연수에서는 어떤 플래그도 켜지 않는다. 강사용 한 줄: "에이전트 팀"은 별도의 실험 기능이며(플래그 이름 `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS`는 확인됨) 오늘은 사용하지 않는다. 대화형 세션에서만 동작하고 헤드리스(`-p`)에서는 팀원이 생성되지 않는다. 막힌 참가자는 짝 화면을 함께 보게 한다. (2분)

도식 설명 2단계 `.flow` (폴더 만들기 → `claude` 실행 확인). 오른쪽에 작은 체크박스 "설정 파일 만지지 않기".

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

25

슬라이드 25 / 71

실습: 플러그인 설치 (설치 통로 열고 설치하기)

실습 5분

- 저장소를 "마켓플레이스"로 등록한다
- 그 안의 `harness`를 설치한다
- `/plugin`에서 `harness`가 enabled면 성공
- 주의: 커뮤니티가 만든 서드파티. 학교 관리 PC는 담당 부서와 먼저 상의

프롬프트 · 파일

Claude Code 입력창에서 (Git 필수: 24번에서 `git --version` 확인):

```
/plugin marketplace add revfactory/harness
/plugin install harness@harness-marketplace
```

확인: `/plugin`

예상 결과

`/plugin` 패널 목록에 `harness`가 보이고 활성화(enabled) 상태.

막힐 때

- `Failed to clone marketplace repository: Command 'git' not found`가 나온다 → git이 없는 것이다. Windows는 Git for Windows를 설치하고 터미널과 Claude Code를 새로 연 뒤 첫 줄부터 다시 한다. 설치 전에 `git --version`으로 확인하면 피할 수 있다.
- 설치 명령이 인식되지 않는다 → 마켓플레이스 이름이 다를 수 있다. `/plugin` 패널에서 등록된 마켓플레이스 이름을 보고 `harness@<그 이름>`으로 바꾼다.
- 학교 PC라서 설치가 막힌다 → 짝 화면을 함께 본다.
- `/plugin install`을 입력했더니 바로 설치되지 않고 패널이 열린다 → 정상이다. 패널에서 설치 범위를 고르고 설치한다.

해설 · 발표자 노트

소요 시간 5분

마켓플레이스 등록은 GitHub 저장소를 git으로 내려받으므로 git이 필수다(공식 문서: Windows에서 git이 없으면 `Failed to clone marketplace repository: Command 'git' not found`, 해결은 Git for Windows 설치). 세션 안에서 `/plugin install`을 입력하면 설치 범위를 고르는 패널이 열린다(이 패널을 따라가는 시간 때문에 5분). 플러그인 저장소를 "마켓플레이스"로 등록하는 것이 첫 줄, 그 안의 플러그인을 설치하는 것이 둘째 줄이다. 마켓플레이스는 플러그인 카탈로그 저장소이므로, 저장소를 먼저 등록하고 그 안의 플러그인을 설치한다. `revfactory/harness`는 Anthropic이 아니라 개인 개발자가 공개한 오픈소스이고, 플러그인 안의 스킬 문구는 Claude Code가 내 권한으로 실행하므로 낯선 저장소는 설치 전에 코드를 훑어보는 습관이 안전하다. 학교 공용 계정이나 관리 PC에서는 설치 전에 담당 부서와 상의한다(원본 문장 유지). 마켓플레이스 이름 `harness-marketplace`와 두 줄 설치 명령은 검수에서 저장소 기준으로 확인됨. (5분)

도식 설명 2줄 명령 + 아래에 `/plugin` 패널 캡처 자리 표시("harness: enabled").

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

---

26 슬라이드 26 / 71

실습 5분

실습: 말로 팀 만들기 (플랜 모드)

- 플랜 모드로 어떤 파일이 만들어질지 먼저 확인: Shift+Tab을 눌러 `⏸ plan mode on`이 보이게 한다(Auto로 시작했다면 세 번). 또는 프롬프트 앞에 `/plan`
- 계획이 나오면 선택지에서 **"Yes, manually approve edits"** 를 고른다 ("Yes, and use auto mode"는 고르지 않는다)
- 하고 싶은 말을 그냥 문장으로 설명한다
- 만드는 에이전트 1 + 검수 에이전트 1 (검수는 읽기만)

프롬프트 · 파일

```
형성평가 문항을 만들고 검수하는 하네스를 구성해줘. 주제를 입력하면
객관식 퀴즈를 생성하는 에이전트 하나와, 생성된 문항의 정답·오탈자만
검수하는 에이전트 하나로 구성해줘. 검수 에이전트는 파일을 수정하지
않고 읽기만 하게 해줘
```

예상 결과

아래와 비슷한 파일 구조(실행마다 파일 이름과 문구가 조금씩 다르다. 예시 중 하나).

생성된 폴더 구조 (예시)

```
mini-harness/
└── .claude/
    ├── agents/
    │   ├── quiz-writer.md      ← 퀴즈 생성 담당
    │   └── quiz-checker.md      ← 검수 담당 (Read만 허용)
    └── skills/
        └── quiz-format/
            └── SKILL.md          ← 문항 형식 절차서
```

막힐 때

- 플러그인이 반응하지 않고 일반 답변만 한다 → 문장 맨 앞이 "하네스를 구성해줘"로 읽히도록 위 문장을 그대로 붙여 넣는다.
- 파일 이름이 나와 다르다 → 정상이다. 에이전트 2개와 스킬 1개가 있으면 된다.
- 계획 승인 화면의 선택지가 다르게 보인다 → 최신 버전은 "Yes, and use auto mode / Yes, manually approve edits / No, keep planning" 세 가지다. 두 번째를 고른다.

해설 · 발표자 노트

소요 시간 5분

이 플러그인은 `/quiz`처럼 정해진 커맨드가 아니라, 문장이 "팀을 구성해 달라"는 요청임을 Claude가 스스로 알아보고 스킬을 실행한다. 처음에는 여러 파일을 한꺼번에 만들므로 플랜 모드(M2에서 배운 Shift+Tab)로 계획을 확인하고 승인하는 습관을 들인다(파일 수정 전 계획 확인). 생성된 `quiz-checker.md`를 열어 보면 `name`, `description`, `tools: Read`, "하지 않는 일"이 이미 들어 있다. 파일 내용은 플러그인이 대신 써 준 것이고, M5에서 열어 보고 말로 고친다고 예고한다. 최신 Claude Code는 Auto 모드로 시작할 수 있고, 계획 승인 화면의 선택지는 "Yes, and use auto mode / Yes, manually approve edits / No, keep planning"이다. 첫 번째를 고르면 세션이 Auto 모드(분류 모델이 대신 검토)로 바뀌므로 연수에서는 "manually approve edits"를 고르게 한다. 상태 표시줄 문구 `⏸ plan mode on`은 확인됨. 새로 만들어진 `.claude` 폴더를 Claude Code가 인식하지 못하면 `/exit` 후 `claude`로 다시 시작한다(세션 시작 때 없던 agents/skills 폴더는 재시작이 필요할 수 있다). (5분)

도식 설명 입력 문장 → 플랜 모드 계획 확인 → 승인(Yes, manually approve edits) → `.claude/` 폴더 생성. 계획 확인 단계에 "읽기 전용 계획 확인" 표식.

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

27 실습: 팀 실행하고 검수 결과 보기

슬라이드 27 / 71

실습 5분

- 만드는 에이전트와 검수 에이전트가 따로 일한다
- 검수 에이전트가 문항의 문제를 지적하는지 확인한다

프롬프트 · 파일

> 이진 탐색 개념으로 객관식 퀴즈 3문항을 만들고 검수까지 맡겨줘

예상 결과

예시 터미널(내용은 실행마다 다름)

```
quiz-writer가 문항을 생성했습니다. quiz-checker를 호출합니다…
[quiz-checker] 2번 문항: 보기 ③과 ④가 모두 조건을 만족해 정답이
두 개로 해석될 수 있습니다. 보기 수정을 권장합니다.
```

막힐 때

- 검수 에이전트가 호출되지 않는다 → 프롬프트 끝의 "검수까지 맡겨줘"를 빼먹지 않았는지 확인하고 다시 보낸다. 안 되면 `.claude/agents/` 폴더에 두 파일(quiz-writer, quiz-checker)이 있는지 확인한다.
- 지적 사항이 없다고 나온다 → 정상일 수 있다. 주제를 "이진 탐색 vs 선형 탐색"처럼 헷갈리기 쉬운 것으로 바꿔 다시 요청해 본다.

해설 · 발표자 노트

소요 시간 5분

주제는 담당 과목에 맞게 바꿔도 된다. 핵심은 "만드는 역할과 검수하는 역할이 서로 다른 컨텍스트에서 실행됐다"는 점을 눈으로 확인하는 것이다. 지적 내용은 예시이고 실제 출력과 달라도 괜찮다. 시간이 남는 참가자는 내용 확인 후 "내가 보기엔 이 지적이 맞나?"를 직접 판단해 본다(사람의 눈도 최종 검수). 끝나면 손 들기: "검수 에이전트의 지적이 나온 분?" `/agents`는 최신 버전에서 목록을 보여 주지 않고 관리 안내만 출력하므로(예전 버전은 목록·생성 화면) 에이전트 확인은 폴더 열기로 한다. (5분)

도식 설명 quiz-writer(문항 생성) → quiz-checker(별도 컨텍스트, 읽기만) → 지적 사항, 화살표 흐름. checker 칸에 "다른 눈" 라벨.

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

28 정리: 하네스가 만든 것

슬라이드 28 / 71

정리 2분

- 하네스가 만든 것: `mini-harness/.claude/` (에이전트 2개 + 스킬 1개)
- 에이전트 = 역할이 정해진 하위 작업자 파일 `.claude/agents/*.md`
- 스킬 = 재사용 절차 `.claude/skills/*/SKILL.md`

- 오케스트레이터 = 팀 실행 순서를 정한 스킬
- 다음 M5: 이 파일을 열어 읽고, 말로 고친다

예상 결과

참가자 폴더에 `mini-harness/.claude/`(에이전트 2개 + 스킬 1개)가 있다.

> **막힐 때**
>
> 플러그인 설치 또는 팀 생성이 안 된 경우 짝 화면을 함께 보며 따라간다.

해설 · 발표자 노트

소요 시간 2분

방금 경험한 것은 만드는 역할과 검수하는 역할이 나뉜 팀을 말로 요청해 파일로 받고 실행한 것이다. 퀴즈를 만든 쪽과 검수한 쪽이 서로 다른 컨텍스트에서 실행되므로 한 에이전트가 "내 답이 맞겠지" 하고 넘어가는 문제가 줄어든다. 구성 요소 이름은 정의 한 줄만 말한다. 이 연수는 이 파일들을 처음부터 설계하는 법이 아니라, 하네스가 만들어 준 파일을 읽고 말로 고치는 법을 다룬다. 다음 M5 예고: 방금 만든 `mini-harness/.claude/`를 열어 보고, 검수 기준을 말로 고쳐 다시 실행한다. `/harness:evolve`는 심화 선택 박스로 한 줄만 소개하고 넘어간다. (2분)

도식 설명 왼쪽 폴더 트리(`mini-harness/.claude/agents/quiz-writer.md`, `quiz-checker.md`, `skills/quiz-format/SKILL.md`), 오른쪽에 세 구성 요소 이름과 한 줄 정의를 해당 파일 옆에 연결선으로 표시. 오케스트레이터는 "실행 순서를 정한 스킬 파일" 한 줄로만 표시. 맨 아래 작은 칸 `심화 선택`: "`/harness:evolve` = 팀을 고친 뒤 처음 설계와 최종 형태가 어떻게 달라졌는지 기록하는 명령. 지금은 이런 게 있다는 정도만."

MODULE M5

모듈 M5

하네스가 만든 파일 읽고 고치기

학습 목표

> 하네스가 만든 `.claude/` 파일을 열어 읽고, 말로 고쳐 다시 실행해 결과를 비교할 수 있다.

시간	**23분**
슬라이드	**29-36 (8장)**
실습 산출물	**고친 `.claude/agents/*.md` + 내 업무용 새 팀**

이 모듈의 슬라이드

번호	제목	유형	시간	쪽
29	하네스가 만든 폴더 지도	개념	2분	·
30	실습: 생성된 파일 열어 보고 구조 설명받기	실습	3분	·
31	실습: 말로 고치기	실습	4분	·
32	실습: 고친 팀 다시 실행해 비교하기	실습	3분	·
33	실습: 내 업무용 새 팀을 하네스에게 요청하기	실습	4분	·
34	점검: 하네스가 만든 것, 사람이 확인할 것	점검	1분	·
35	정리: 하네스로 만들고, 말로 고치고, 다시 실행	정리	1분	·
36	휴식 5분	정리	5분	·

29

슬라이드 29 / 71

하네스가 만든 폴더 지도

개념 2분 확인 필요

- 하네스가 만든 팀은 `.claude/` 폴더 안의 **마크다운 파일**이다. 파일을 열면 팀의 구성이 그대로 보인다.
- 에이전트 = 역할이 정해진 하위 작업자를 정의한 파일(`.claude/agents/*.md`, 한 파일에 한 에이전트)
- 스킬 = 재사용하는 작업 절차를 적은 파일(`.claude/skills/이름/SKILL.md`)
- 오케스트레이터 = 팀의 실행 순서를 정한 스킬
- 오늘은 이 파일들을 **읽고, 말로 고치고, 다시 실행**한다. 처음부터 쓰지 않는다.

도식 · 단계

```
mini-harness/
└── .claude/
    ├── agents/
    │   ├── quiz-writer.md    ← 에이전트: 문항을 만드는 역할
    │   └── quiz-checker.md   ← 에이전트: 문항을 검수하는 역할
    └── skills/
        └── quiz-.../         ← 오케스트레이터 스킬: writer → checker 순서를 정한 절차 (예)
            └── SKILL.md
```

트리 아래 주석: "파일 이름과 개수는 하네스가 실행마다 정한다. 위는 예시."

해설 · 발표자 노트

소요 시간 2분

M4에서 하네스가 만든 파일의 위치를 지도로 한 번 보여 주는 슬라이드다. 구성 요소 정의는 한 줄씩만 말하고 원리 설명으로 들어가지 않는다. 핵심 메시지는 "하네스가 만든 것은 눈에 보이는 일반 파일이므로 열어서 읽고 말로 고칠 수 있다"이다. 파일 이름은 M4 26·27번 예시와 맞췄고 실행마다 다를 수 있다고 말한다 확인 필요. (2분)

도식 설명 폴더 트리 + 파일별 한 줄 정의 (이 슬라이드의 유일한 도식). 왼쪽에 트리, 오른쪽에 각 줄의 한 줄 정의를 화살표로 연결.

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 오케스트레이터 스킬 폴더의 실제 이름

30

슬라이드 30 / 71

실습: 생성된 파일 열어 보고 구조 설명받기

실습 3분 확인 필요

- M4에서 팀을 만든 폴더에서 Claude Code를 연다 (`cd mini-harness` → `claude`)
- `!`로 시작하는 입력은 Claude Code 안에서 터미널 명령을 그대로 실행한다
- 목록을 확인한 뒤, Claude Code에게 구조를 설명해 달라고 한다

프롬프트 · 파일

Claude Code 입력창에서 목록 확인:

```
! ls .claude/agents .claude/skills
```

이어서:

```
.claude 폴더에 있는 에이전트와 스킬 파일을 읽고, 이 팀의 구조를 3줄로 설명해줘. 누가 무엇을 하는지와 실행 순서가 들어가게 해줘
```

예상 결과

`agents`에 `quiz-writer.md`, `quiz-checker.md` 같은 파일이, `skills`에 폴더가 하나 이상 보인다(예: 이름은 실행마다 다름). 설명은 "writer가 문항 생성 → checker가 검수 → 결과 정리" 같은 3줄이다.

> **막힐 때**
>
> - `ls`에 아무것도 안 나온다 → 팀을 만든 폴더가 아니다. `! pwd`로 위치를 확인하고 M4에서 쓴 폴더로 이동해 `claude`를 다시 연다.
> - `.claude` 폴더는 있는데 Claude Code가 에이전트를 모른다고 한다 → `/exit` 후 `claude`로 재시작한다 확인 필요 .
> - 설명이 3줄보다 길다 → 정상이다. 그대로 읽고 다음으로 간다.

해설 · 발표자 노트

소요 시간 3분

목록은 사람이 직접 확인하고 구조 이해는 Claude Code에게 맡긴다. 파일을 열고 싶은 참가자는 에디터로 `quiz-checker.md`를 열어 봐도 된다. 읽을 때 볼 것은 "역할 설명이 내 수업 맥락과 맞는가" 하나뿐이다. 문법이나 각 항목의 의미는 설명하지 않는다. (3분)

도식 설명 2단계 가로 흐름: `! ls` 로 파일 목록 확인 → 말로 "3줄 설명" 요청.

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 재시작 필요 여부

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

31

슬라이드 31 / 71

실습: 말로 고치기

실습 4분 확인 필요

- 파일을 직접 편집하지 않는다. **고치고 싶은 점을 문장으로** 말한다.
- 두 가지를 요청한다: ① 검수 기준 한 줄 추가 ② 만드는 문항 수 변경
- 바뀐 곳만 보여 달라고 해서 **무엇이 바뀌었는지** 눈으로 확인한다

프롬프트 · 파일

```
quiz-checker의 검수 기준에 "정답이 둘 이상으로 해석되는 보기는 반드시 지적한다"를 한 줄 추가하고, quiz-writer가 만드는 기본 문항 수를 5개로 바꿔줘. 파일을 고치기 전에 무엇을 바꿀지 먼저 알려줘
```

수정이 끝나면:

```
방금 바꾼 줄만 보여줘
```

예상 결과

두 파일에서 각각 한두 줄이 추가·변경된 모습이 나온다(예: 검수 기준 목록에 한 줄 추가, 문항 수 숫자 3 → 5). 고친 파일 이름은 실제 생성된 이름을 따른다.

막힐 때

- 파일 수정을 승인하라는 화면이 나온다 → 정상이다. 바뀔 내용을 읽고 승인한다.
- 기본 문항 수가 파일에 적혀 있지 않다고 한다 → 정상일 수 있다. "문항 수는 요청에서 정하게 두고, 따로 없으면 5개를 기본으로 한다는 문장을 추가해줘"라고 이어서 말한다.
- 고친 뒤에도 다음 실행에서 예전 동작이 나온다 → `/exit` 후 `claude`로 재시작한다 확인 필요.
- 오류 메시지가 나온다 → 읽으려 하지 말고 그대로 Claude Code에게 붙여 넣어 "이 오류를 고쳐줘"라고 한다.

해설 · 발표자 노트

소요 시간 4분

이 슬라이드의 핵심은 "말로 요청하면 하네스가 만든 파일이 고쳐지고, 바뀐 곳은 확인할 수 있다"는 흐름이다. 사람이 하는 일은 승인과 확인이다. 검수 기준 문장은 자기 교과에 맞게 바꿔도 된다(예: "코드 예시는 실행 결과까지 확인한다"). (4분)

도식 설명 3단계 흐름: 말로 요청 → Claude Code가 파일 수정(승인) → "바뀐 줄만 보여줘"로 확인.

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 재시작 필요 여부

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

---

32 슬라이드 32 / 71

실습: 고친 팀 다시 실행해 비교하기

실습 3분 확인 필요

- M4 27번에서 쓴 요청과 같은 문장으로 다시 실행한다
- 이전 결과와 비교: 문항 수, 검수 지적의 달라진 점

프롬프트 · 파일

```
이진 탐색 개념으로 객관식 퀴즈를 만들고 검수까지 맡겨줘
```

예상 결과

문항이 5개로 나오고, 정답이 둘로 해석될 수 있는 보기가 있으면 검수 에이전트가 지적한다(예: 내용은 실행마다 다름). 지적이 없으면 검수 기준에 걸리는 문항이 없었다는 뜻일 수 있다.

막힐 때

- 문항이 여전히 3개다 → `/exit` 후 `claude`로 재시작하고 다시 보낸다 확인 필요. 그래도 같으면 "quiz-writer.md의 문항 수 부분을 보여줘"라고 해서 고쳐진 내용을 확인한다.
- 검수 에이전트가 호출되지 않는다 → 문장 끝의 "검수까지 맡겨줘"를 그대로 쓴다. `.claude/agents/`에 `quiz-checker.md`가 있는지 확인한다.

해설 · 발표자 노트

소요 시간 3분

고친 말이 실행 결과에 반영되는 것을 눈으로 확인하는 단계다. 결과 비교는 완벽한 일치가 아니라 "달라진 점이 있는가"만 본다. 출력은 실행마다 다르다. 시간이 남으면 "지적이 맞는지 내 판단은?"을 짝과 이야기한다. (3분)

도식 설명 좌우 2열 비교표. 왼쪽 "M4 실행 결과(3문항)", 오른쪽 "고친 뒤 결과(5문항)". 비교 항목: 문항 수 / 검수 지적 / 정답 해석.

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 재시작 필요 여부

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

슬라이드 33 / 71

33 실습: 내 업무용 새 팀을 하네스에게 요청하기

실습 4분 확인 필요

- M4 26번과 같은 방식: **"하네스를 구성해줘" + 하고 싶은 일**을 문장으로 설명
- 플랜 모드로 계획을 확인하고 승인한다 (`Shift+Tab`, 선택지는 "Yes, manually approve edits")
- 오늘은 수업안 작성 도우미를 예로 든다. 내 업무에 맞게 바꿔도 된다.

프롬프트 · 파일

새 폴더에서 `claude`를 열고 (예: `mkdir lesson-helper && cd lesson-helper && claude`):

```
하네스를 구성해줘. 단원과 학습 목표를 주면 수업안 초안을 쓰는 역할과, 초안이 학습 목표와 맞는지 검수하는 역할로 구성해줘. 검수는 파일을 고치지 않고 읽기만 해줘
```

예상 결과

계획 화면에 만들 파일 목록이 나오고, 승인하면 `.claude/agents/`에 에이전트 파일 두 개와 `.claude/skills/`에 절차 스킬이 생긴다(예: 이름·개수는 실행마다 다름).

막힐 때

- 하네스가 아니라 일반 답변만 한다 → 플러그인 설치를 확인한다(M4 25번). 문장 맨 앞의 "하네스를 구성해줘"는 그대로 둔다.
- 파일이 생겼는데 슬래시나 호출이 인식되지 않는다 → `/exit` 후 `claude`로 재시작한다 확인 필요.
- 계획 승인 선택지가 다르게 보인다 → "Yes, manually approve edits"에 해당하는 항목을 고른다.

해설 · 발표자 노트

소요 시간 4분

슬라이드 31의 "말로 고치기"를 새 팀에도 쓸 수 있음을 보여 주는 슬라이드다. 학생 실명·성적 같은 개인정보는 요청 문장에 넣지 않는다. 만들어진 팀을 실행하는 단계는 시간상 하지 않고 M7 프로젝트에서 본격적으로 한다. 파일이 생성되는 데 시간이 걸리는 참가자는 짝 화면을 함께 본다. (4분)

도식 설명 입력 문장 → 플랜 모드 계획 확인 → 승인 → `.claude/` 아래에 파일 추가, 3단계 가로 흐름. M4 26번 도식의 축약판.

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 재시작 필요 여부

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

34

점검 1분

점검: 하네스가 만든 것, 사람이 확인할 것

- [ ] 만들어진 내용(퀴즈·수업안)이 **내 교과 내용으로 맞는가**
- [ ] 검수 기준이 **내가 중요하게 보는 기준과 맞는가** (맞지 않으면 말로 고친다)
- [ ] 파일과 요청 문장에 **학생 개인정보가 없는가**

예상 결과

세 항목을 모두 확인했다.

막힐 때

기준이 애매하면 슬라이드 31의 방식으로 검수 기준을 말로 추가한다.

해설 · 발표자 노트

소요 시간 1분

사람이 확인할 것은 문법이나 파일 형식이 아니라 "무엇이 만들어졌는지"와 "내 교과 내용이 맞는지"다. 하네스가 만든 검수 에이전트가 있어도 교과 내용의 최종 판단은 교사의 몫이다. (1분)

도식 설명 체크박스 3개 카드 가로 배열.

점검 메모

35 슬라이드 35 / 71 정리: 하네스로 만들고, 말로 고치고, 다시 실행

정리 1분

- 흐름: 말로 요청(M4) → 생성 파일 읽기 → 말로 고치기 → 다시 실행해 비교
- 고칠 때는 파일을 직접 편집하지 않고 Claude Code에게 말한다. 오류는 그대로 붙여 넣는다.
- 다음: 10분 휴식 후 **Firebase 기능 지도(M6)**

해설 · 발표자 노트

소요 시간 1분

오늘 익힌 흐름은 M7 프로젝트에서 같은 방식으로 쓴다(단원 명세를 쓰고 하네스에게 팀 구성을 요청). 휴식 슬라이드는 별도(33)이다. (1분)

도식 설명 4칸 가로 흐름(요청 → 읽기 → 고치기 → 실행). 맨 오른쪽에 화살표로 "다음: Firebase(M6)".

36 슬라이드 36 / 71 휴식 5분

정리 5분 확인 필요

- 5분 쉬어 갑니다.
- 막힌 분은 강사석으로 오세요.
- 휴식 중에 Firebase 콘솔(console.firebase.google.com)에 개인 구글 계정으로 로그인해 두세요.
- 다음 모듈(M6)은 Firebase 기능 지도입니다. 요금제를 Blaze로 바꾸는 단계가 있으니 결제 수단을 준비해 두세요(본인 개인 계정).

해설 · 발표자 노트

소요 시간 5분

휴식 슬라이드이다. 설명/실습 시간 합계에 넣지 않는다. 막힌 참가자를 한 명씩 확인하고 짝 화면을 함께 보게 한다. 휴식 뒤 다음 모듈(M6, Firebase 기능 지도)에 대비해 휴식 중에 Firebase 콘솔(console.firebase.google.com)에 개인 구글 계정으로 로그인해 두도록 안내한다. 학교 관리 계정(Workspace)은 프로젝트 생성이 막혀 있을 수 있으니 개인 계정을 쓴다 확인 필요. 실제 학생 정보는 쓰지 않는다. (5분, 휴식)

도식 설명 큰 타이머 표시 "5:00".

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 상세 없음

MODULE M6

모듈 M6

Firebase 기능 지도와 콘솔 설정

학습 목표

> Firebase 6개 항목을 언제 쓰는지 고르고, 콘솔과 CLI로 인증·데이터베이스·요금제·Storage·Cloud Functions 설정을 함께 해 볼 수 있다. 접근 규칙은 Claude Code가 만든다.

시간	**32분**
슬라이드	**37-55 (19장)**
실습 산출물	**인증(Google 로그인)·Firestore·요금제(Blaze, 예산 알림)·Storage 설정과 Functions 배포를 마친 Firebase 테스트 프로젝트**

이 모듈의 슬라이드

번호	제목	유형	시간	쪽
37	Firebase 기능 지도: 앱 백엔드 서비스 묶음	개념	1분	·
38	공통 설정 1: 테스트 프로젝트 만들기	실습	3분	·
39	공통 설정 2: 웹 앱 등록과 firebaseConfig	실습	2분	·
40	Authentication 사용 상황: 사용자 인증과 uid	개념	1분	·
41	Authentication 설정 절차: Google 로그인 켜기	실습	3분	·
42	Firestore 사용 상황: 문서 지향 NoSQL 데이터베이스	개념	1분	·
43	Firestore 설정 절차: 데이터베이스 만들기	실습	3분	·
44	요금제 사용 상황: Spark와 Blaze	개념	1분	·
45	요금제 설정 절차: Blaze로 바꾸고 예산 알림 걸기	실습	2분	·
46	Storage 사용 상황: 파일(객체) 저장	개념	1분	·
47	질문: Firestore냐, Storage냐, 올리지 않을까	점검	1분	·
48	Storage 설정 절차: 버킷 만들기	실습	2분	·
49	Hosting 사용 상황: 정적 웹 콘텐츠 호스팅	개념	1분	·
50	Hosting 설정 절차: Node.js·CLI 설치와 로그인	실습	2분	·
51	질문: 배포 버튼을 누르기 전, 사람이 꼭 볼 것 3가지는?	점검	1분	·
52	Cloud Functions 사용 상황: 서버리스 백엔드 코드	개념	1분	·
53	Cloud Functions 설정 절차: 함수 배포와 비밀값	실습	4분	·
54	종합 상황 카드: 필요한 기능과 요금제는?	실습	1분	·
55	설정 체크리스트와 정리	정리	1분	·

37 Firebase 기능 지도: 앱 백엔드 서비스 묶음

슬라이드 37 / 71

개념 1분

- Firebase = Google의 앱 백엔드 플랫폼. 여러 서비스의 묶음이며 필요한 서비스만 골라 쓴다.
- 오늘 6개 항목: Authentication, Firestore, 요금제, Storage, Hosting, Cloud Functions
- 무료(Spark)로 되는 것과 결제 수단(Blaze)이 필요한 것이 나뉜다. 6개 모두 오늘 함께 설정해 본다.

도식 · 단계

기능	역할	요금제 표식	클래스룸 예시 한 단어
Authentication	사용자 인증, uid 발급	Spark	로그인
Firestore	문서 지향 NoSQL 데이터베이스	Spark	수업 기록
요금제(Spark/Blaze)	Spark = 무료 요금제 / Blaze = 종량제	—	선택
Storage	파일(객체) 저장	**Blaze 필요**	큰 파일
Hosting	정적 웹 콘텐츠 호스팅, 공개 URL	Spark	공개 주소
Cloud Functions	서버리스 백엔드 코드(비밀 키 사용 가능)	**Blaze 필요**	AI 피드백

표 아래 범례: 함께 해 볼 것 = Authentication · Firestore · 요금제 · Storage · Hosting · Cloud Functions.
표 아래 한 줄: 접근을 제한하는 보안 규칙은 이 연수에서 설명하지 않는다. Claude Code가 작성하고 배포한다(M7).

해설 · 발표자 노트

소요 시간 1분

오늘의 학습 목표는 "6개를 언제 쓰는지 고르고, 인증·DB·요금제·Storage·Functions를 설정해 본다"이다. 지도를 먼저 보여 주고, 6개 모두 함께 직접 설정해 본다고 예고한다. Blaze가 필요한 요금제·Storage·Cloud Functions는 결제 수단이 필요하므로 45번에서 예산 알림부터 정한다고 미리 말한다. 접근 규칙은 설명하지 않고 Claude Code가 만든다는 점을 한 줄만 말한다. Spark/Blaze 표식은 사실표 8절 기준이다. Storage 표식은 "2026-02-03부터 Blaze 필요"이다(검수 확인: pricing 페이지 Spark의 Cloud Storage가 "Not applicable", Storage FAQ와 일치). Hosting이 Spark로 되는 이유는 사실표 8절. 클래스룸은 예시 소재일 뿐 앱을 만들지 않는다고 한 번 더 말한다. (1분)

도식 설명 6개 카드가 한 장에 놓인 지도. 카드마다 `[역할 한 줄] / 기능 이름 / 요금제 표식 / 클래스룸 예시 한 단어`.

38 공통 설정 1: 테스트 프로젝트 만들기

슬라이드 38 / 71

실습 참가자 실습 Spark 콘솔 직접 조작 3분 확인 필요

- Firebase 프로젝트 = 서비스·설정·앱을 묶는 최상위 단위. 하나 만들어 보자.
- 이름에 **test**를 넣는다. 실제 학생 정보는 넣지 않는다.

도식 · 단계

☐ **1.** console.firebase.google.com 에 개인 구글 계정으로 로그인

☐ **2.** **프로젝트 만들기(추가)** 버튼 클릭 확인 필요

☐ **3.** 프로젝트 이름 입력(예: `classroom-test-이니셜`, **실명은 넣지 않는다**) → 약관 확인 → 계속

- 이름은 내가 알아보려고 붙이는 표시이고, 나중에 바꿀 수 있다.
- 이름과 별개로 Firebase가 고유한 **프로젝트 ID**를 자동으로 정해 준다. ID는 만든 뒤에는 바꿀 수 없고, `PROJECT_ID.web.app` 같은 공개 주소에 드러난다. 영문 소문자·숫자·하이픈만 쓴다. M7에서 이 ID를 Claude Code에 알려 준다.

☐ **4.** Gemini in Firebase, Google Analytics 단계가 나오면 연수에서는 **끄거나 기본값** 확인 필요 (Analytics는 끄기 권장, 작성자 판단). Gemini를 켜더라도 **Gemini API 키는 절대 코드에 넣지 않는다**(39번)

☐ **5.** **프로젝트 만들기(Create project)** 클릭 → 프로젝트 개요 화면이 뜨면 성공

예상 결과

프로젝트 개요(Overview) 페이지가 열려 있고 상단에 내 프로젝트 이름이 보인다.

> **막힐 때**
>
> - 프로젝트 만들기가 막히거나 권한 오류 → 학교 관리(Workspace) 계정일 수 있다. 개인 구글 계정으로 다시 로그인 확인 필요.
> - 버튼 문구가 다르면 화면의 "프로젝트 추가/만들기"에 해당하는 버튼을 고른다. 강사 화면과 짝 화면을 본다.

해설 · 발표자 노트

소요 시간 3분

프로젝트는 Authentication, Firestore 같은 서비스와 그 설정, 등록된 앱을 묶는 최상위 단위다. 사실표 1절 기준. 프로젝트 생성은 Spark(무료)로 시작한다. 이미 만든 프로젝트에 기능만 추가할 때는 새 프로젝트를 만들지 않는다(사실표 1절). 사실표는 9단계였으나 5단계로 압축했다(웹 앱 등록은 39번으로 이동). 버튼 문구와 Gemini/Analytics 화면 순서는 사실표 11절 1번에 따라 확인 필요. 연수 후에 이 프로젝트를 삭제하는 방법은 55번에서 안내한다. 예시 이름에 실명을 넣지 않는 이유: 프로젝트 ID가 공개 주소에 드러난다(검수 7-D). 이름과 ID의 관계(공식 문서 https://firebase.google.com/docs/projects/learn-more): 프로젝트 이름은 여러 프로젝트를 구별하려는 내부 표시라 언제든 바꿀 수 있다. 프로젝트 ID는 Firebase가 고유하게 자동 지정하고, 프로젝트를 설정하는 동안에는 고칠 수 있지만 리소스가 만들어진 뒤에는 바꿀 수 없다. 문서에는 ID가 이름에서 만들어진다는 설명이 없으므로 "이름에서 만들어진다"고 말하지 않고, 화면에 ID가 보이거나 고칠 수 있는 칸이 나오면 거기에도 실명이 없는지 같이 본다 확인 필요. 공개 주소가 `PROJECT_ID.web.app` 형태라는 점은 M7 배포 주소와 이어진다. Gemini in Firebase를 켜는 선택지가 나오므로 "Gemini 같은 AI 서비스 키는 비밀"이라는 점을 39번과 연결해 한 줄 말한다. (3분)

도식 설명 `.steps` 5단계

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 정확한 버튼 문구
- 화면 순서
- 학교 계정에서 생성이 막히는지
- 상세 없음
- 화면에서 ID가 보이는 위치

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

---

39 공통 설정 2: 웹 앱 등록과 firebaseConfig

슬라이드 39 / 71

실습 | 참가자 실습 | Spark | 콘솔 직접 조작 | 2분 | 확인 필요

- 웹 앱 등록 = 프로젝트에 웹 앱 구성(firebaseConfig)을 생성하는 일.

- **firebaseConfig(안에 든 apiKey 포함) = 클라이언트가 프로젝트를 식별하는 설정값. 공개되어도 된다. 접근 통제는 Claude Code가 만드는 보안 규칙이 맡는다(M7).**
- **AI 서비스 키(Gemini 등)·서비스 계정 키 = 서버 측 자격증명. 저장소·클라이언트 코드에 노출하면 안 된다.** (서비스 계정 키 = 서버가 관리자 권한으로 접속할 때 쓰는 자격증명 파일, JSON)

도식 · 단계

☐ 1. 프로젝트 개요 페이지에서 **웹 아이콘(`</>`)** 클릭

☐ 2. **앱 닉네임** 입력(내부 구분용 이름일 뿐)

☐ 3. **앱 등록(Register app)** 클릭

☐ 4. 화면에 나오는 `firebaseConfig` 를 **눈으로 확인**만 한다(복사해서 붙일 앱은 만들지 않는다)

비교 카드:

- 공개해도 OK (프로젝트 식별 설정값): `firebaseConfig` 전체, 그 안의 `apiKey` 포함. 프로젝트를 식별만 하고 권한은 주지 않는다.
- 절대 공개 금지 (서버 측 자격증명): 서비스 계정 키(JSON), AI 서비스 키(Gemini 등), 외부 서비스 비밀 키

예상 결과

`firebaseConfig` 코드 블록이 화면에 보인다. 항목은 apiKey, authDomain, projectId 등이다 확인 필요.

> 막힐 때
>
> - 웹 아이콘이 안 보임 → 프로젝트 개요 페이지로 돌아가 "앱 추가" 영역을 찾는다 확인 필요.
> - "설정값을 숨겨야 하나?" → 숨기지 않아도 된다. 접근 통제는 Claude Code가 만드는 보안 규칙이 맡는다(M7).

해설 · 발표자 노트

소요 시간 2분

공개 가능 근거는 공식 문서(https://firebase.google.com/docs/projects/api-keys, 검수 확인): 키는 프로젝트와 앱을 **식별**할 뿐 **권한 부여**는 하지 않는다. **안전한 조건: 그 키가 Firebase 서비스로만 제한되어 있어야 하고, 허용 목록에 Generative Language API(Gemini)가 없어야 한다.** 그래서 Gemini API 키는 절대 코드에 넣지 않는다. 서비스 계정 키·AI 서비스 키도 코드·HTML·깃에 넣지 않는다. 참가자는 화면에서 `apiKey` 라는 글자를 직접 보므로, "API 키는 모두 비밀"이라고 말하지 않는다(구분: firebaseConfig = 프로젝트 식별 설정값, AI 서비스 키·서비스 계정 키 = 서버 측 자격증명). 흔한 오류: 설정값을 비밀로 오해해 숨기다 앱이 안 돌아가는 것. 접근 규칙의 내용은 이 연수에서 설명하지 않고 M7에서 Claude Code가 만든다. 이 단계는 앱을 만드는 것이 아니라 "접속 정보가 어떻게 생겼는지 보는" 단계다. (2분)

도식 설명 `.steps` 4단계 + 오른쪽에 2칸 비교 카드

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 항목 전체 목록은 문서 본문에서 미확인, 일반적으로 알려진 형태
- 화면 위치

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

40

슬라이드 40 / 71

Authentication 사용 상황: 사용자 인증과 uid

개념 1분

- Authentication = 사용자 인증 서비스. 로그인 공급자(예: 구글 계정)로 사용자를 확인하고 고유 식별자(uid)를 발급한다.
- 쓴다: ① 내 학생만 들어오게 ② 학생마다 자기 기록만 보기
- 안 쓴다: 누구나 보는 공지 페이지

해설 · 발표자 노트

소요 시간 1분

사실표 2절. 쓰는 상황은 "학생·교사별로 보이는 화면과 권한을 나눌 때", "학생 제출물을 본인 것으로 저장할 때". 안 쓰는 상황은 "로그인이 필요 없는 공개 안내 페이지". 클래스룸 예시에서는 구글 로그인으로 교사/학생을 구분한다(예시일 뿐). 요금 참고: 기본 인증은 월간 활성 사용자 5만 명까지 무료(Spark·Blaze 동일, 사실표 2절). 이 로그인으로 받은 uid가 사용자를 구분하는 기준이 되며, 구분에 따른 접근 조건은 M7에서 Claude Code가 만든다고 예고만 한다. (1분)

도식 설명 3칸. `쓰는 상황 1` / `쓰는 상황 2` / `안 쓰는 상황` (회색 카드). 가운데에 사용자 → Authentication(로그인 공급자 확인) → uid 발급.

41

슬라이드 41 / 71

Authentication 설정 절차: Google 로그인 켜기

실습 참가자 실습 Spark 콘솔 직접 조작 3분 확인 필요

- 콘솔에서 Google 로그인 방법을 켠다.
- 마지막에 **승인된 도메인** 목록을 눈으로 확인한다.

도식 · 단계

- [ ] 1. 콘솔 왼쪽 메뉴 **보안(Security) > Authentication** 확인 필요
- [ ] 2. **시작하기(Get started) → 로그인 방법(Sign-in method)** 탭 확인 필요
- [ ] 3. **Google** 공급자 선택 → **사용 설정**
- [ ] 4. **프로젝트 지원 이메일** 선택 → **저장** 확인 필요
- [ ] 5. **설정(Settings) 탭 > 승인된 도메인(Authorized domains)** 목록을 눈으로 확인한다. 2025-04-28 이후 만든 프로젝트는 기본 목록에 localhost가 없다. M6에서는 추가하지 않는다. 배포 주소에서 로그인 오류가 나면 그때 승인된 도메인을 확인한다(M7 64). 확인 필요

예상 결과

로그인 방법 목록에서 **Google**이 "사용 설정됨"으로 표시된다.

막힐 때

- 메뉴를 못 찾음 → 왼쪽 메뉴 그룹 이름이 보안(Security)인지 확인한다. 안 보이면 빌드(Build) 그룹을 연다 확인 필요.
- 나중에 앱에서 `auth/unauthorized-domain` 오류 → 지금 접속한 주소가 승인된 도메인 목록에 없다는 뜻이다. 목록에 추가한다(사실표 2절).

해설 · 발표자 노트

소요 시간 3분

사실표 2절. 승인된 도메인은 "로그인 팝업을 허용하는 사이트 주소 목록"이다. 개발 중 도메인은 자동 추가되나 직접 연결한 도메인은 추가해야 한다. **검수 확인(Auth FAQ, 이메일 링크 문서):** "2025-04-28 이후 만든 프로젝트의 승인된 도메인 기본값에는 localhost가 없고, 설정(Settings) 탭의 승인된 도메인(Authorized domains)에서 추가한다." Google은 운영 프로젝트에 localhost를 쓰지 말라고 권한다. M6에서는 추가하지 않는다. 배포 주소에서 로그인 오류가 나면 그때 승인된 도메인을 확인한다(M7 64). 이미 localhost가 있으면 운영 때 지우도록 권장한다. 로그인 방법 탭·Google 사용 설정은 google-signin 문서로 확인됨. 코드 예시(`signInWithPopup`)는 이 연수에서 쓰지 않는다. 앱을 만들지 않기 때문이다. 지금은 Google 로그인을 사용 설정하는 데까지만 한다. 초보 실습에서는 `signInWithRedirect`보다 `signInWithPopup`이 안전하다는 점은 사실표 2절의 참고 사항(심화). (3분)

도식 설명 `.steps` 5단계

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 한국어 메뉴명, 리허설에서 실제 화면과 대조
- 시작하기 버튼 문구
- 지원 이메일 항목은 문서 본문 미확인
- Hosting 기본 주소의 기본 포함 여부
- 상세 없음

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

42

슬라이드 42 / 71

개념 1분

Firestore 사용 상황: 문서 지향 NoSQL 데이터베이스

- Firestore = 문서 지향 NoSQL 데이터베이스. **데이터는 컬렉션 안의 문서에 저장하고, 문서는 필드(키-값)로 이루어진다.**
- 쓴다: ① 수업·학습지·제출물 저장 ② 교사가 올린 자료를 학생이 실시간으로 보기
- 안 쓴다: 큰 파일(영상·큰 이미지) → Storage가 맞다. 단 Storage는 Blaze 필요.

해설 · 발표자 노트

소요 시간 1분

사실표 3절. 한 줄 정의는 "문서를 컬렉션에 담아 저장하는 실시간 클라우드 데이터베이스". 용량 한도: 문서 1개 최대 1MiB(검수 확인: quotas 문서 "Maximum size for a document 1 MiB (1,048,576 bytes)"). MiB·GiB는 저장 용량 단위(1MiB는 약 1MB)라고 한 줄 덧붙인다. 짧은 글(문항·답안·HTML 문자열)은 Firestore 문서에 두면 결제 수단 없이 가능하다(사실표 5·9절). 요금 참고: Standard Edition 무료 한도 1GiB 저장, 하루 읽기 5만, 쓰기 2만(Spark). 컬렉션 그룹 쿼리·복합 색인은 심화이므로 여기서는 이름만 언급하고 넘어간다(필요하면 사실표 3절 "색인" 참고). (1분)

도식 설명 컬렉션-문서-필드 계층도 하나. 컬렉션 2개 `users/{uid}`, `classes/{classId}` 아래에 문서와 필드. (예시 경로는 이 두 개만. 클래스룸 예시 소재)

43

슬라이드 43 / 71

Firestore 설정 절차: 데이터베이스 만들기

실습 | 참가자 실습 | Spark | 콘솔 직접 조작 | 3분 | 확인 필요

- 데이터베이스를 만들 때 **프로덕션(잠금) 모드**로 시작한다.
- 프로덕션 모드는 처음에 모든 접근을 막고, 접근 조건은 Claude Code가 규칙 파일로 만들어 배포한다(M7).
- 테스트 모드는 누구나 읽기·쓰기가 되는 기한이 정해진 임시 설정이라 쓰지 않는다.

도식 · 단계

☐ **1.** 프로젝트 선택 → 왼쪽 패널 **데이터베이스 및 스토리지(Databases & Storage) > Firestore** 확인 필요

☐ **2.** **데이터베이스 만들기(Create database)** 클릭

☐ **3.** 에디션 선택 화면이 나오면 **Standard** 선택. 에디션 = 데이터베이스 종류, 기본형을 고른다. 확인 필요

☐ **4.** **위치(location)** 선택. 서울(asia-northeast3)을 고를 수 있다. **위치는 만든 뒤 바꿀 수 없으므로 신중히 고른다.**

☐ **5.** 보안 모드 **프로덕션 모드** 선택 → **만들기(Create)**

구분	테스트 모드	프로덕션 모드
접근	누구나 읽기·쓰기 허용	클라이언트(웹/앱) 접근 전부 거부
기간	기한이 정해진 임시 설정 확인 필요	만료 없음

예상 결과

Firestore 데이터 화면이 열린다. 데이터는 비어 있다.

> **막힐 때**
>
> - 위치를 고를 수 없음 → 프로젝트 기본 Google Cloud 리소스 위치가 이미 정해진 경우다(사실표 3절). 그대로 진행한다.
> - 나중에 `Missing or insufficient permissions` → 프로덕션 모드는 처음에 전부 거부한다. 정상이다. 접근 조건은 M7에서 Claude Code가 규칙 파일로 만들어 배포한다.

해설 · 발표자 노트

소요 시간 3분

사실표 3절. **프로덕션 모드 권장은 사실표의 작성자 의견**이다. 테스트 모드는 "연수 첫 체험용으로만, 학생 정보가 들어가면 위험". 테스트 모드의 30일 만료는 현재 quickstart 문서에서 문장을 찾지 못해 "기한이 정해진 임시 설정"으로 말하고 확인 필요로 둔다(검수 5절). 확인된 것: 프로덕션 모드는 "Denies all reads and writes from mobile and web clients"(quickstart), 위치는 "once you provision a database instance, you cannot change its location"(locations 문서), 서울(asia-northeast3)은 locations 목록에 있다. Cloud Firestore API는 만들 때 자동 사용 설정된다. 규칙 문법은 이 연수에서 다루지 않는다. 이 단계에서 데이터를 넣거나 앱을 연결하지 않는다. (3분)

도식 설명 `.steps` 5단계 + 아래 비교 표(2열)

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 한국어 메뉴명, 리허설에서 실제 화면과 대조
- 에디션 선택 화면 유무와 화면 순서
- 30일 만료
- 상세 없음

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

44

슬라이드 44 / 71

요금제 사용 상황: Spark와 Blaze

개념 1분 확인 필요

- **Spark = 무료 요금제**(무료 사용량 한도 안에서만 사용). **Blaze = 종량제 요금제**(무료 한도 안은 무료, 넘으면 사용량만큼 청구, 결제 수단 등록 필수).
- Blaze 필요: Cloud Functions 배포, Cloud Storage(2026-02-03부터)
- 용어: MAU = 월간 활성 사용자 수 / GiB = 저장 용량 단위 / GB-초 = 함수 실행 시간 × 메모리

도식 · 단계

기능	Spark	Blaze
Authentication(기본)	월 5만 MAU	동일
Firestore Standard	1GiB, 읽기 5만/일, 쓰기 2만/일	동일 무료 한도 후 종량
Hosting	10GB 저장, 360MB/일 전송	동일 무료 한도
Storage	사용 불가	5GB 저장 등 표기
Functions	사용 불가	월 200만 호출, 40만 GB-초

해설 · 발표자 노트

소요 시간 1분

사실표 8절. 공식 이름: "No-cost (Spark plan)", "Pay as you go (Blaze plan)". 기본 Authentication, Firestore, Hosting은 Spark에서 사용 가능. Blaze가 필요한 것: Cloud Functions(배포), Cloud Storage(2026-02-03부터). Storage는 Spark에서 기존 버킷 포함 읽기/쓰기 접근을 잃고 새 기본 버킷도 Blaze에서만 생성 가능하다. **충돌 없음(검수 확인)**: pricing 페이지 Spark의 Cloud Storage가 "Not applicable"로 바뀌었고, FAQ의 Blaze 요건은 2026-02-03 시행이며 미업그레이드 시 데이터는 남지만 접근할 수 없다. 무료 사용량·한도(Auth 5만 MAU, Firestore 1GiB·읽기 5만·쓰기 2만, Hosting 10GB·360MB/일, Functions Spark 불가)는 pricing 페이지로 확인됨. 무료 사용량 수치는 문서 표기이며 바뀔 수 있으므로 연수 당일 pricing 페이지(https://firebase.google.com/pricing)로 다시 확인한다. Blaze여도 무료 사용량 안에서는 비용이 0일 수 있으나 **결제 수단 등록은 필수**다. 신규 사용자에게 $300 체험 크레딧이 적용되는지는 확인 필요. 그래서 이 연수는 45번에서 모두 Blaze로 올려 함께 실행하고, 예산 알림을 먼저 건다. 연수 후 정리 방법은 55번이다. (1분)

도식 설명 2열 비교 표(문서 표기)

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 상세 없음

45

슬라이드 45 / 71

요금제 설정 절차: Blaze로 바꾸고 예산 알림 걸기

실습 참가자 실습 Blaze 2분 확인 필요

- **예산 알림은 알림일 뿐, 사용을 멈추지 않는다.** 그래서 Blaze로 바꾸기 전에 예산 알림부터 정한다.
- 결제 수단은 **내 개인 계정**에 **내가 직접** 등록한다. 학교 카드나 다른 사람의 카드는 쓰지 않는다.

도식 · 단계

☐ 1. 콘솔 **설정(Settings) > 사용량 및 결제(Usage and billing)** 로 이동 확인 필요

☐ 2. 요금제를 **Blaze로 변경**하고 결제 수단을 등록한다 확인 필요

☐ 3. 업그레이드 화면에서 제안되는 **알림 전용 예산(Alerts-only budget)**의 금액과 알림 임계값(예: 50%, 90%, 100%)을 정한다 확인 필요

☐ 4. 요금제가 Blaze로 바뀌었는지, 예산이 저장되었는지 화면에서 확인한다

- 예산 유형: **알림 전용**(이메일만, 서비스 계속) / **지출 한도(Spend cap)**(Firebase AI Logic, App Hosting, Cloud Functions, Extensions 한정, 100% 도달 시 그 달 남은 기간 중지)

예상 결과

사용량 및 결제 화면에 요금제 Blaze와 내가 정한 예산이 보인다.

> **막힐 때**
>
> - 결제 수단 등록이 안 되거나 카드가 거절됨 → 그 단계에서 멈추고 짝 화면을 보며 Storage(45)와 Cloud Functions(50)는 화면으로 따라간다. 억지로 다른 카드를 쓰지 않는다 확인 필요.
> - 메뉴를 못 찾음 → 설정(톱니 아이콘)에서 사용량 및 결제를 찾는다 확인 필요.

해설 · 발표자 노트

소요 시간 2분

이 단계는 개인 결제와 직접 관련되므로 강사도 대신 눌러 주지 않고, 참가자가 화면을 읽고 본인이 직접 확인하게 한다. 사실표 8절 핵심 문구(공식): "Budget alerts do **not** pause services." Firestore·Storage·Hosting에는 지출 한도를 걸 수 없다(문서에 나열된 제품에 미포함). Cloud Functions에는 지출 한도 예산을 걸 수 있다. 제품별 사용량 대시보드가 있다. Blaze여도 무료 사용량 안에서는 비용이 0일 수 있으나 **결제 수단 등록은 필수**이고, 신규 사용자에게 $300 체험 크레딧이 적용되는지는 확인 필요다. 연수가 끝난 뒤 정리 방법은 55번에 있다(테스트 프로젝트 삭제, 청구 내역 확인). 설정 위치와 한국어 메뉴명은 리허설 때 실화면으로 대조한다. (2분)

도식 설명 `.steps` 4단계

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 한국어 메뉴명
- 화면 문구와 순서
- 임계값 기본값
- 상세 없음

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

46

슬라이드 46 / 71

Storage 사용 상황: 파일(객체) 저장

개념 1분

- Storage = 파일(객체) 저장 서비스. 이미지·PDF·영상 같은 **파일**을 저장한다.
- 쓴다: ① 교사가 PDF·이미지·영상 자료 업로드 ② 학생이 사진 과제 제출
- 안 쓴다: 짧은 글·HTML 텍스트 → Firestore (문서 한도 1MiB)

해설 · 발표자 노트

소요 시간 1분

사실표 5절. **현재 요건: Storage 사용에는 Blaze 필요, 2026-02-03부터 적용**(검수 확인: Storage FAQ). 문서 1MiB 한도는 quotas 문서로 확인됨. 그래서 이번 클래스룸 예시는 HTML을 Firestore 문서에 저장하는 방식을 우선 권장한다(결제 수단 불필요). 학생 사진은 개인정보이므로 연수에서는 올리지 않는다(47번 질문과 연결). 설계서에 적힌 "2024-10-30 이후 정책"은 사실표에서 확인되지 않았고 사실표는 2026-02-03 요건을 최신으로 본다. (1분)

도식 설명 Storage(파일 저장)와 Firestore(문서 저장) 두 박스. 큰 파일은 Storage로, 짧은 글은 Firestore로 화살표. Storage에 `Blaze 필요` 표식.

47

슬라이드 47 / 71

질문: Firestore냐, Storage냐, 올리지 않을까

점검 참가자 질문 1분

- **학습지 문항은? 수업 동영상은? 학생 사진은?**
- 짝과 1분 이야기 → Firestore / Storage / 올리지 않음

해설 · 발표자 노트

소요 시간 1분

모범 답안. 학습지 문항 = 짧은 글이므로 **Firestore**. 수업 동영상 = 큰 파일이므로 **Storage**(Blaze 필요). 학생 사진 = **개인정보이므로 올리지 않는다**(연수에서는 실제 학생 정보 금지, 실제 운영 시에도 학교 방침·동의가 먼저). 정답 맞히기가 목적이 아니라 "짧은 글은 Firestore, 파일은 Storage, 개인정보는 일단 멈춘다"를 판단하는 연습이다. (1분)

도식 설명 3개 질문 카드 × 3개 답 선택지(Firestore · Storage · 올리지 않음)

점검 메모

48

슬라이드 48 / 71

Storage 설정 절차: 버킷 만들기

실습 참가자 실습 Blaze 2분 확인 필요

- 메뉴 → 시작하기 → 위치 → 접근 규칙 확인. 요금제는 45번에서 Blaze로 바꿨다.
- 접근 규칙은 Claude Code가 만든 규칙 파일을 쓴다. 이 연수에서는 파일을 올리지 않는다.

도식 · 단계

☐ 1. 콘솔 **데이터베이스 및 스토리지(Databases & Storage) > Storage** 확인 필요

☐ 2. **시작하기(Get started)** 클릭. 요금제가 Spark면 Blaze 업그레이드 화면이 먼저 나온다 확인 필요

☐ 3. 버킷 **위치** 선택

☐ 4. 접근 규칙(Claude Code가 만든 규칙 파일)이 배포되었는지 Claude Code에게 확인시킨다. 개발용 공개 접근 옵션이 있으나 **학생 파일에는 쓰지 않는다**

☐ 5. 완료. 파일 올리기와 웹 코드 연결은 이 연수에서 하지 않는다

예상 결과

Storage 화면에 버킷이 만들어져 있고, 파일 목록은 비어 있다.

> **막힐 때**
>
> - Spark 상태라 시작하기가 막힘 → 45번에서 Blaze로 바꿨는지 확인한다. 바꾸지 못했다면 짝 화면으로 따라간다.
> - 버킷 이름이 두 가지로 보임 → 예전 `*.appspot.com` 버킷과 새 `PROJECT_ID.firebasestorage.app`의 차이다. 어느 쪽이든 연수에서는 사용하지 않는다.

해설 · 발표자 노트

소요 시간 2분

사실표 5절. 흔한 오류: Spark 프로젝트에서 접근 불가·버킷 생성 불가(Blaze 필요), 버킷 이름 혼동(기존 `*.appspot.com` 버킷 vs 새 `PROJECT_ID.firebasestorage.app`). 버킷 = Storage 안의 파일 보관 단위(Storage 안의 최상위 저장 단위). 무료 사용량은 Blaze에서도 계속 제공된다(기존 기본 버킷 5GB 저장 등, 새 버킷은 Google Cloud Storage "Always Free"가 US-CENTRAL1, US-EAST1, US-WEST1 한정, 다른 리전은 일반 요금). 연수 계정의 $300 체험 크레딧 적용 여부는 확인 필요. 출시 전 점검 항목으로 App Check, 예산 알림, 사용량 대시보드 확인을 공식 문서가 안내한다. 규칙의 내용은 설명하지 않는다. 무료 한도가 있는 US 리전 외 위치를 고르면 요금이 날 수 있으므로 위치 선택 때 한 번 짚는다 확인 필요. (2분)

도식 설명 `.steps` 5단계

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 한국어 메뉴명
- 업그레이드 화면 흐름
- 상세 없음
- 리전별 요금

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

49

슬라이드 49 / 71

Hosting 사용 상황: 정적 웹 콘텐츠 호스팅

개념 1분

- Hosting = 정적 웹 콘텐츠 호스팅 서비스. 만든 웹페이지를 공개 URL로 제공한다.
- 쓴다: ① 학생이 링크로 접속하는 웹 자료 공개 ② 수정 후 다시 배포해 즉시 반영
- 안 쓴다: 나 혼자 로컬에서 테스트만 할 때, 학생 개인정보가 든 파일

해설 · 발표자 노트

소요 시간 1분

사실표 6절. 기본 HTTPS 제공. 요금 참고: 저장 10GB, 전송 하루 360MB 무료(Spark·Blaze 동일). 사용자 지정 도메인은 선택 사항. Firebase Hosting은 Spark에서 사용 가능하다(사실표 8절). **Firebase App Hosting과의 차이(검수 확인)**: App Hosting은 서버 렌더링 풀스택 웹앱용이고 Blaze가 필요하다("upgrade to the pay-as-you-go Blaze pricing plan to use App Hosting"). Hosting은 정적·단일 페이지 웹을 CDN으로 배포한다. 슬라이드에는 한 줄만 덧붙인다. 개인정보 파일 금지는 설계서 지시이며 공개 주소에 올라가면 누구나 볼 수 있기 때문이다. (1분)

도식 설명 배포 파일 → Hosting → `PROJECT_ID.web.app` 공개 URL 흐름. 아래에 "개인정보 파일은 붙이지 않는다" 경고 라벨.

50

슬라이드 50 / 71

Hosting 설정 절차: Node.js·CLI 설치와 로그인

실습 참가자 실습 Spark 2분

확인 필요

- 터미널에서 4단계: CLI 설치 → 로그인 → 초기화 → 배포. 앞의 두 단계를 지금 함께 한다.
- 배포는 **공개 행위**다. Claude Code에게는 **플랜 모드**로 시킨다. 초기화와 배포는 M7(63번)에서 Claude Code가 한다.

도식 · 단계

- [ ] **1.** Node.js(CLI를 돌리는 실행 환경) 확인: `node --version`. 없으면 설치한다 확인 필요
- [ ] **2.** `npm install -g firebase-tools` 후 `firebase --version` (Firebase CLI = 터미널용 명령 프로그램)
- [ ] **3.** `firebase login` (브라우저에서 구글 계정 인증)
- [ ] **4.** Claude Code에게 배포 **계획만** 요청해 본다(아래 요청, 실행하지 않는다)
- [ ] **5.** 초기화 `firebase init hosting`과 배포 `firebase deploy --only hosting`은 M7(63번)에서 한다

- Claude Code에 시키는 요청 문장 예시(플랜 모드에서):

```
이 폴더를 Firebase Hosting에 배포하는 단계를 계획만 세워줘. 실행은 하지 말고,
내가 승인한 뒤에만 명령을 실행해. 배포 전에 공개 폴더에 개인정보가 든 파일이 없는지 먼저 확인해줘.
```

예상 결과

`node --version`과 `firebase --version`에 버전이 나오고, `firebase login`이 끝나면 터미널에 로그인 완료 문구가 보인다.

막힐 때

- `npm`이 없다고 나옴 → Node.js가 설치되지 않은 것이다. Node.js를 설치하고 터미널을 새로 연다.
- 브라우저가 안 열림 → `firebase login --no-localhost`를 실행하고 화면의 안내대로 코드를 붙여 넣는다 확인 필요.
- Docker 경로(M1 5~7번)로 시작함 → Node.js·Firebase CLI는 이미지에 들어 있으므로 1~2단계를 건너뛴다. 3단계는 새 PowerShell에서 `docker compose run --rm claude firebase login --no-localhost`로 하고, 주소를 PC 브라우저에서 열어 나온 코드를 붙여 넣는다. 로그인은 보관되어 이후 Claude Code 안에서도 쓰인다 확인 필요.
- 학교 PC라서 설치가 막힘 → 짝 화면을 함께 본다. 62번에서 Claude Code에게 설치 방법을 물어볼 수도 있다 확인 필요.

해설 · 발표자 노트

소요 시간 2분

사실표 6절. 이 단계에서 설치한 Node.js·CLI와 `firebase login`은 M7 62번(Firebase 연결)에서 그대로 쓴다. 62번에서는 `firebase projects:list`로 로그인만 확인하면 된다. 흔한 오류: 로그인 안 된 상태에서 배포, 공개 폴더 안에 `index.html` 없음(기본 안내 페이지가 뜸), 배포 주소를 승인된 도메인에 추가하지 않아 `auth/unauthorized-domain`(기본 주소는 이미 포함인지 확인 필요), `.firebaserc`의 잘못된 프로젝트 선택. `firebase init hosting`, `firebase deploy --only hosting`, `web.app`·`firebaseapp.com` 주소는 Hosting quickstart로 확인됨(검수). 질문 순서만 미확인. 요청 문장 예시는 배포 **계획**만 요청하며 앱 코드를 만들지 않는다. 배포는 사람이 직접 확인한 뒤에 한다(51번). (2분)

도식 설명 `.steps` 5단계

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 지원 버전, 학교 PC 설치 권한
- 상세 없음
- Windows 리허설
- 학교 PC 설치 권한

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

---

51 슬라이드 51 / 71

질문: 배포 버튼을 누르기 전, 사람이 꼭 볼 것 3가지는?

점검 참가자 질문 1분

- **배포 전에 사람이 꼭 볼 것 3가지는?**
- 힌트: 접근 규칙, 개인정보, 공개 범위

해설 · 발표자 노트

소요 시간 1분

모범 답안. ① **접근 규칙(Claude Code가 만든 규칙 파일)이 배포되었는지 Claude Code에게 확인시킨다**(접근 규칙 없이 공개하면 데이터가 노출됨, 사실표 1절 흔한 오류) ② **개인정보가 공개 폴더에 들어 있지 않은지** ③ **공개 범위**(누가 이 주소를 볼 수 있는가). 이 세 가지는 사람이 직접 확인한다. M3의 "실제 배포에 완전 자동 승인 금지"와도 연결한다. 시간 부족 시 짝과 30초만 말해 보게 한다. (1분)

도식 설명 빈칸 3개 카드(①②③)와 힌트 접힘 상자. 오른쪽에 사람이 확인하는 단계 아이콘.

점검 메모

---

52 슬라이드 52 / 71 Cloud Functions 사용 상황: 서버리스 백엔드 코드

개념 1분 확인 필요

- Cloud Functions = 서버리스 백엔드 코드 실행 서비스. 클라이언트(브라우저)가 직접 하면 안 되는 일을 **서버 쪽에서 대신 처리**한다.
- 쓴다: ① AI API 키를 서버에 숨기고 AI 피드백 생성 ② 제출되면 자동으로 후처리
- 안 쓴다: 저장·조회만 하면 되는 일(Firestore만으로 충분)

해설 · 발표자 노트

소요 시간 1분

사실표 7절. 한 줄 정의는 "서버를 직접 두지 않고 필요할 때만 실행되는 서버 쪽 코드를 올려 두는 서비스". 웹에서 부르는 함수는 callable(`onCall`)을 쓰며 Firebase Auth 토큰이 자동 전달되어 로그인 사용자를 확인할 수 있다. 요금: **배포에는 Blaze 필요**(공식 문서 확인). 2세대 무료 사용량(Blaze): 월 200만 호출, 40만 GB-초. AI 호출 비용은 AI 제공업체의 별도 요금이며 확인 필요. (1분)

도식 설명 학생 브라우저 → Cloud Functions(서버 측, Secret으로 저장된 AI 키 사용) → AI 서비스. 브라우저에는 키가 없음을 표시.

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 연수에서 사용할 AI 서비스 확정 후

---

53 슬라이드 53 / 71 Cloud Functions 설정 절차: 함수 배포와 비밀값

실습 참가자 실습 Blaze 4분

확인 필요

- Blaze 확인 → CLI 초기화 → **비밀값 저장** → 배포. 연수용 함수 하나로 흐름만 따라 한다.
- 비밀값(AI API 키)은 브라우저 코드·HTML·`.env` 커밋에 **넣지 않는다**. 지금은 **가짜 값**으로 연습한다.

도식 · 단계

☐ **1.** 요금제가 Blaze이고 예산 알림이 있는지 확인한다 (45번)

☐ **2.** 새 폴더 `functions-test`에서 `firebase init functions` (프로젝트 선택, 언어 선택 등 질문에 답함)
확인 필요

☐ **3.** 아래 요청으로 연수용 함수 하나를 만든다(플랜 모드, 배포는 하지 않는다)

☐ **4.** 비밀값을 **Secret**으로 저장하는 연습: `firebase functions:secrets:set TEST_SECRET`, 값은 `test-secret` 같은 **가짜 값**

☐ **5.** 배포 `firebase deploy --only functions` → 계획을 읽고 승인, 콘솔 Functions 목록에서 확인

- 3단계에서 Claude Code(플랜 모드)에 붙여 넣는 요청:

```
functions 폴더에 hello라는 이름의 HTTP 함수를 하나만 만들어줘. 비밀값은 쓰지 마.
계획만 보여주고, 내가 승인하기 전에는 배포하지 마.
```

예상 결과

콘솔 Functions 목록에 `hello` 함수가 보인다. 실행할 때마다 결과가 조금 다를 수 있다.

막힐 때

- 배포가 거부됨 → Blaze가 아닌 것이다. 45번을 다시 확인한다.
- `Secret` 관련 오류로 배포가 막힘 → 값을 입력했는지 확인한다. 이 연습에서는 함수에 비밀값을 연결하지 않는다.
- 시간이 부족함 → 3~5단계는 짝 화면을 보며 따라간다. 끝낸 사람은 55번 정리를 먼저 읽는다.

해설 · 발표자 노트

소요 시간 4분

사실표 7절. **이 슬라이드는 실행 대기 때문에 4분을 넘기기 쉬우므로 리허설에서 소요 시간을 재 둔다.** 연수용 함수는 클래스룸 앱 코드가 아니라 배포 흐름을 보기 위한 최소 예시이며, 실제 AI API 키는 입력하지 않는다. 비밀값 관리: 권장 방식은 매개변수화된 구성(Parameterized configuration)이며 유형에 Secret이 포함된다. Secret 값은 Cloud Secret Manager에 저장되고 **개별 함수에 바인딩**해야 쓸 수 있다. 위 명령과 코드의 `defineSecret`·`secrets: [...]` 옵션은 functions get-started, config-env 문서로 확인됨(검수). 이 연습은 저장까지만 하고 함수에 연결하지 않는다. 일반(비밀 아닌) 설정은 `functions/.env`. 옛 방식 `functions.config()`는 deprecated(v6.0.0), 2027년 3월 중단 예정이므로 **연수 자료에서 쓰지 않는다**. 흔한 오류: Blaze가 아니라 배포 거부, Secret 값 누락으로 CLI가 배포를 막음, Node 버전 불일치, Secret을 함수에 바인딩하지 않아 런타임 값이 빔. `firebase init functions`가 만드는 샘플 코드의 상태는 확인 필요이므로 함수는 Claude Code에게 요청해 만든다. 배포한 함수는 연수 후 정리 대상이다(55번). (4분)

도식 설명 `.steps` 5단계

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 질문 순서와 문구
- 상세 없음

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

54

슬라이드 54 / 71

종합 상황 카드: 필요한 기능과 요금제는?

실습 참가자 실습 1분 확인 필요

- 카드 3장을 보고 **먼저 직접** 필요한 기능을 고른다.
- 그다음 Claude Code에게 같은 질문을 하고 비교한다. **링크를 확인하기 전에는 믿지 않는다.**

도식 · 단계

☐ **1.** "PDF·이미지·영상 같은 큰 파일을 올리고 싶다"

☐ **2.** "만든 앱을 학생에게 링크로 공개하고 싶다"

☐ **3.** "AI로 제출 답안에 맞춤 피드백을 달고 싶다(API 키는 숨기고 싶다)"

아래에 내 답 칸: `기능 ______ / 요금제 Spark·Blaze`

프롬프트 · 파일

```
이 수업 아이디어에 필요한 Firebase 기능과 요금제(Spark/Blaze)를 표로 정리해줘.
각 행에 근거가 되는 Firebase 공식 문서 링크를 같이 적어줘. 확실하지 않은 것은 "확인필요"라고 적어줘.
앱 코드는 만들지 마.

수업 아이디어: (위 카드 중 하나 또는 내가 만들고 싶은 수업 아이디어를 적는다)
```

예상 결과

기능과 요금제가 표로 나오고 행마다 공식 문서 링크가 붙는다. 사실표 9절의 정답과 비교하면 ① Storage(Blaze 필요) ② Hosting(Spark) ③ Cloud Functions + Secret(Blaze 필요).

막힐 때

- 링크가 열리지 않거나 내용이 다름 → 그 행은 믿지 않는다. 링크를 직접 열어 확인하고, 확인 못 하면 확인 필요로 남긴다.
- 요금제가 내 답과 다름 → 44번 표와 pricing 페이지(https://firebase.google.com/pricing)를 기준으로 다시 본다.

해설 · 발표자 노트

소요 시간 1분

AI가 낸 표는 근거 링크를 사람이 열어 확인한 뒤에야 쓴다. 카드 3장은 사실표 9절 선택표에서 가져왔고 정답은 사실표 그대로이다(접근 통제 카드는 규칙을 설명하지 않는 방침에 따라 뺐다). 선택표에서 Storage는 Blaze 필요, Hosting은 Spark(`web.app` 주소), Functions+Secret은 Blaze 필요이다. 프롬프트에 앱 코드를 만들지 말라는 문장을 넣었다. 시간이 없으면 카드 1장만 다룬다. (1분)

도식 설명 상황 카드 3장(사실표 9절 선택표에서 가져옴)

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 상세 없음

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

55 설정 체크리스트와 정리

슬라이드 55 / 71

정리 실습 시간 1분 확인 필요

- 오늘 함께 해 본 것을 표시한다.
- 연수가 끝나면 **테스트 프로젝트를 삭제**하고, 결제 계정의 청구 내역을 한 번 확인한다. M7에서 만든 워크북을 계속 쓸 사람은 삭제하지 않는다.

도식 · 단계

함께 해 봤다 (Spark로 가능)	함께 해 봤다 (Blaze 필요·CLI)
☐ 프로젝트 만들기	☐ 요금제 Blaze 변경·예산 알림
☐ 웹 앱 등록, firebaseConfig 확인	☐ Storage 시작하기
☐ Google 로그인 켜기	☐ Node.js·CLI 설치와 `firebase login`
☐ Firestore 만들기(프로덕션 모드)	☐ Cloud Functions 배포

하단: 연수 후 삭제 → 콘솔 **프로젝트 설정 > 일반(General) > 맨 아래 "프로젝트 삭제"** → 체크박스를 확인하고 삭제. 30일 안에는 Google Cloud 콘솔에서 복구할 수 있다. 삭제하지 않고 계속 쓰는 사람은 사용량 대시보드와 예산 알림을 확인하고, 쓰지 않는 Functions와 Storage는 정리한다.

해설 · 발표자 노트

소요 시간 1분

프로젝트 삭제 절차는 검수에서 확인되었다(Firebase 도움말 "Delete your project", https://support.google.com/firebase/answer/9137886). 한국어 UI 메뉴명은 리허설에서 한 번 대조한다. 실제 학생 계정과 이름은 입력하지 않았는지 마지막으로 확인한다. 이 모듈 전체의 비용 원칙: 연수에서는 45번에서 Blaze로 올리고 예산 알림을 먼저 건다. 연수 후에는 테스트 프로젝트를 삭제하거나 쓰지 않는 서비스를 정리하고, 다음 달까지 청구 내역과 예산 알림 이메일을 확인한다. 삭제 후 결제 계정 연결이 어떻게 되는지, Blaze에서 Spark로 되돌리는 방법은 공식 문서로 확인하지 못했다 확인 필요. (1분)

도식 설명 체크표 2열

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 상세 없음

MODULE M7

모듈 M7

프로젝트: 우리 교과 단원 수업 패키지 만들기

학습 목표

> 자기 교과 단원의 명세를 쓰고 하네스로 수업 슬라이드와 학생 워크북 웹앱을 만들어 Firebase Hosting 주소로 확인할 수 있다.

시간	**50분**
슬라이드	**56-67 (12장)**
실습 산출물	`unit-project/` (`unit-spec.md`, `.claude/`, `slides.html`, `workbook/`, `firestore.rules`) **+ 배포 주소**

이 모듈의 슬라이드

번호	제목	유형	시간	쪽
56	오늘의 프로젝트: 우리 교과 단원 수업 패키지	개념	2분	·
57	단원 명세 쓰기: unit-spec.md	실습	5분	·
58	하네스에게 팀 구성 요청하기	실습	5분	·
59	수업 슬라이드 만들기: 하네스 팀 실행 1단계	실습	7분	·
60	사람이 확인: 교과 내용과 학습 목표	점검	3분	·
61	터미널에서 한 번 로그인하면 Claude Code가 이어받는다	개념	2분	·
62	Firebase 연결: 미리 로그인해 두기	실습	4분	·
63	워크북 웹앱 만들고 배포하기	실습	8분	·
64	배포 주소로 확인하기	실습	4분	·
65	고쳐 달라고 말하고 다시 배포하기	실습	5분	·
66	짝과 바꿔 점검하기	점검	3분	·
67	오늘 만든 패키지 한 장 회고	정리	2분	·

슬라이드 56 / 71

56 오늘의 프로젝트: 우리 교과 단원 수업 패키지

개념 2분

- 내 교과의 한 단원으로 **수업 슬라이드(`slides.html`)와 학생 워크북 웹앱**을 만들고, **Firebase Hosting 배포 주소**로 확인하는 것이 최종 목표다.
- 사람은 단원 명세 작성, Firebase 설정(M6에서 완료), 승인만 한다. 팀 구성은 하네스가, 코드·규칙·배포는 Claude Code가 한다.
- 흐름: 단원 명세 → 하네스 구성 → 수업 슬라이드 → 워크북 배포 → 점검·공유. 개인 작업, 마지막에 짝과 바꿔 점검한다.
- 하네스 = 작업을 설명하면 역할이 나뉜 팀(`.claude/agents/`, `.claude/skills/` 파일)을 만들어 주는 플러그인(revfactory/harness, M4에서 설치).
- 지켜야 할 것: 테스트 계정만 쓴다 · 학생 실제 정보는 넣지 않는다 · 교과 내용의 정확성은 교사가 책임진다.
- 테스트 계정 = Firebase 실습에 쓰는 개인 구글 계정(학생·업무 계정이 아닌 것).

해설 · 발표자 노트

소요 시간 2분

"오늘은 앞 모듈에서 배운 것을 하나의 산출물로 묶습니다." 만드는 것은 두 가지다. 하나는 수업에서 넘겨 가며 쓰는 슬라이드 파일, 다른 하나는 학생이 구글 계정으로 로그인해 답을 저장하는 워크북 웹앱이다. 최종 목표는 웹앱을 Firebase Hosting으로 배포하고 주소를 브라우저로 여는 것이다. 순서는 다섯 단계다. 먼저 교사가 단원 명세를 쓰고, 그 명세로 하네스에게 팀 구성을 요청한다. 하네스는 M4에서 설치한 플러그인이고, 오늘은 팀의 내부 구조를 설계하지 않는다. 팀이 만들어지면 슬라이드, 워크북 배포, 점검 순서로 진행한다. 핵심 체험은 하나다. Firebase 설정은 M6에서 이미 했고, 오늘은 로그인 승인과 계획·명령 승인만 하면 나머지는 하네스와 Claude Code가 한다. 규칙 세 가지를 먼저 말한다. 첫째, 테스트용 구글 계정만 쓴다. 둘째, 실제 학생 이름·이메일·응답은 넣지 않는다. 셋째, Claude Code가 쓴 교과 내용은 틀릴 수 있으므로 최종 확인은 교사가 한다. 이 프로젝트는 개인별이고, 짝은 66번에서 서로 바꿔 점검할 때 만난다. 교과는 정보가 아니어도 된다. 예시 단원은 정보 교과의 "알고리즘 — 탐색" 하나로 계속 쓴다. (2분)

도식 설명 가로 흐름 5단계 칸. 왼쪽부터 ① 단원 명세(`unit-spec.md`, 사람이 씀) ② 하네스 구성(팀 생성, 사람이 계획 승인) ③ 수업 슬라이드(`slides.html`) ④ 워크북 배포(`workbook/` + 규칙 + 배포 주소) ⑤ 점검·공유(사람이 확인, 짝 점검, 배포 주소 전달). 칸 아래에 슬라이드 번호 범위: ① 57 ② 58 ③ 59-60 ④ 61-65 ⑤ 66-67. 오른쪽 아래에 작업 폴더 트리 `unit-project/` 아래 `unit-spec.md`(내가 씀), `.claude/agents/` · `.claude/skills/`(하네스가 생성), `slides.html`, `workbook/`, `firestore.rules`, `firebase.json`(Claude Code가 생성). 맨 아래에 안전 원칙 3개 라벨(테스트 계정만 · 실제 학생 정보 없음 · 교과 내용은 교사 확인).

슬라이드 57 / 71

57 단원 명세 쓰기: unit-spec.md

실습 5분

- 새 폴더 `unit-project`에서 `claude`를 시작하고, Claude Code가 **빈 항목마다 질문만** 하게 한다. 내용은 교사가 쓴다.
- 항목: 단원 코드 · 교과·학년·단원명 · 성취기준(교육과정 문서에서 복사) · 학습 목표 2~3 · 핵심 개념 3~5 · 워크북 문항 5~8(객관식·단답·서술) · 출처.
- 이 명세 한 장이 다음 단계에서 하네스가 읽는 입력이다.
- 예시: 정보 교과 "알고리즘 — 탐색". 내 교과로 바꾸려면 교과·단원명·성취기준·핵심 개념·문항만 바꾼다.

프롬프트 · 파일

(먼저 `/exit` → `cd ..` → `mkdir unit-project` → `cd unit-project` → `claude`로 claude-training 밖의 새

폴더에서 시작한 뒤 붙여 넣는다.)

```
unit-spec.md 파일을 만들어줘. 항목은 다음과 같다: 단원 코드(영문 소문자·숫자·하이픈), 교과·학년·단원명, 성취기준(내가 붙여 넣는 문장 그대로), 학습 목표 2~3개, 핵심 개념 3~5개, 워크북 문항 5~8개(객관식·단답·서술을 섞고, 문항마다 id·유형·질문·교사용 정답), 출처.
파일에는 항목 제목만 쓰고 내용은 비워 둬. 그다음 비어 있는 항목을 한 번에 하나씩 나에게 질문해줘.
내가 답한 문장은 그대로 기록하고, 내용을 새로 지어내거나 고치지 마. 내가 모른다고 하면 "확인 필요"라고 적어 둬.
```

예상 결과

`unit-project/unit-spec.md`가 생기고 질문에 답할수록 항목이 채워진다. 아래는 정보 교과 예시로 모두 채운 모습이다. 성취기준과 출처는 교사가 입력한다. 내 화면과 글자가 달라도 항목 7개(단원 코드, 교과·학년·단원명, 성취기준, 학습 목표, 핵심 개념, 워크북 문항, 출처)가 모두 있으면 성공이다.

unit-project/unit-spec.md

```
# 단원 명세

## 단원 코드
info-search-01

## 교과 · 학년 · 단원명
정보 · [교사 입력: 학년] · 알고리즘 — 탐색

## 성취기준
[교사 입력: 교육과정 문서에서 코드와 문장을 그대로 복사]

## 학습 목표
1. 선형 탐색과 이진 탐색의 절차를 순서대로 설명할 수 있다.
2. 이진 탐색을 쓰려면 자료가 정렬되어 있어야 함을 말할 수 있다.
3. 자료의 수가 늘 때 두 탐색의 비교 횟수가 달라지는 이유를 설명할 수 있다.

## 핵심 개념
탐색, 선형 탐색, 이진 탐색, 정렬된 자료, 비교 횟수

## 워크북 문항
- q1 (객관식) 이진 탐색을 쓰려면 자료가 어떤 상태여야 하는가? 보기: 무작위 / 정렬됨 / 중복 없음 / 문자열만. 정답: 정렬됨
- q2 (객관식) 자료 8개에서 선형 탐색으로 찾을 때 최악의 비교 횟수는? 보기: 1 / 3 / 4 / 8. 정답: 8
- q3 (단답) 이진 탐색은 한 번 비교할 때마다 탐색 범위가 어떻게 되는가? 정답: 절반으로 줄어든다
- q4 (단답) 정렬된 자료 15개에서 이진 탐색의 최대 비교 횟수는? (비교 횟수 = 가운데 값과 비교한 횟수) 정답: 4
- q5 (서술) 선형 탐색이 더 알맞은 상황을 하나 들고 이유를 쓰시오. 채점 기준: [교사 입력]
- q6 (서술) 이진 탐색이 선형 탐색보다 빠른 까닭을 비교 횟수로 설명하시오. 채점 기준: [교사 입력]

## 출처
[교사 입력: 교과서 쪽수 또는 자료 이름]
```

막힐 때

- Claude Code가 성취기준·정답을 직접 채워 넣음 → "내가 답하지 않은 항목은 비워 두고 '확인 필요'로 적어줘"라고 다시 요청한다. 성취기준은 교육과정 문서에서 직접 복사한다.
- 한 번에 여러 질문을 함 → "한 번에 하나씩만 질문해줘"라고 요청한다.
- 문항이 5개에 못 미침 → 지금은 5개면 충분하다. 나머지는 연수 후에 `unit-spec.md`에 더한다.
- 5분 안에 못 끝냄 → 예시 file-box를 그대로 `unit-spec.md`로 저장해 달라고 요청하고 진행한다(내 교과 내용은 연수 후 바꾼다).

해설 · 발표자 노트

소요 시간 5분

"여기서부터 5분은 교사가 쓰는 시간입니다." Claude Code는 질문하고 기록하는 역할이다. 내용을 쓰는 사람은 교사다. 이유는 두 가지다. 교과 내용의 정확성을 교사가 책임져야 하고, 이 명세가 다음 단계에서 하네스가 만든 팀이 읽는 유일한 입력이기 때문이다. 명세에 없는 사실은 팀이 근거 없이 채우게 되므로 비워 둘 항목은 "확인 필요"로 남긴다. 성취기준은 요약하지 않고 교육과정 문서에서 그대로 복사해 붙인다. 시간이 없으면 학습 목표 2개, 핵심 개념 3개, 문항 5개로 줄인다. claude-training 밖(`cd ..`)에 새 폴더를 만드는 이유: 이 폴더에 하네스가 `.claude/`를 새로 만들므로, 상위 폴더의 CLAUDE.md와 M4·M5에서 만든 팀이 섞이지 않게 한다. (5분)

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

슬라이드 58 / 71

58 하네스에게 팀 구성 요청하기

실습 5분 확인 필요

- `unit-project`의 같은 세션에서, 단원 명세로 **하네스에게 팀 구성을 요청**한다. 하네스는 M4(25번)에서 설치했다.
- 순서: ① 플랜 모드로 바꾸고(`⏸ plan mode on`이 보일 때까지 Shift+Tab, 또는 붙여 넣을 내용 맨 앞에 `/plan `) 아래 프롬프트를 붙여 넣는다. ② 계획을 읽고 **"Yes, manually approve edits"**로 승인한다. ③ 만들어진 팀 목록을 확인한다. ④ **`/exit` → `claude`로 다시 시작**한다.
- 하네스가 만드는 것: 에이전트(역할이 정해진 하위 작업자, `.claude/agents/*.md`), 스킬(재사용 절차, `.claude/skills/*/SKILL.md`), 오케스트레이터(팀 실행 순서를 정한 스킬). 개수와 이름은 하네스가 정한다.
- 새로 만든 `.claude/agents`, `.claude/skills` 폴더는 세션 시작 때 없었으므로, 재시작해야 인식된다.
- 이 단계에서는 슬라이드와 앱 코드를 만들지 않는다. 팀만 만든다.

프롬프트 · 파일

```
하네스를 구성해줘.
단원 명세는 unit-spec.md야. 만들 것은 ① 수업용 슬라이드 slides.html ② 학생 워크북 웹앱(Firebase Host
ing에 배포) ③ 교과 내용 검수야.
지금은 팀 구성만 하고 slides.html과 앱 코드는 만들지 마.
학생 개인정보와 비밀 키는 다루지 말고, 교사용 정답은 웹앱에 넣지 마.
도구 설치와 명령 실행은 실행 전에 물어봐.
```

예상 결과

플랜 모드가 만들 팀(에이전트와 스킬 목록)을 먼저 보여 준다. 이어서 승인 화면에 세 선택지가 뜬다: "Yes, and use auto mode" / "Yes, manually approve edits" / "No, keep planning". **"Yes, manually approve edits"**를 고른다(첫째를 고르면 세션이 Auto 모드로 바뀌어 파일이 확인 없이 만들어진다). 첫째 선택지가 "Yes, auto-accept edits"로 보일 수도 있다. 승인 뒤 파일마다 승인 창이 뜨면 내용을 보고 Yes를 고른다. 아래는 생성 결과의 한 예다. 이름과 개수는 실행마다 다르다.

unit-project/.claude (생성된 팀의 예)

```
.claude/agents/   에이전트 파일 (예: 슬라이드 담당, 워크북 담당, 검수 담당)
.claude/skills/   스킬 파일 (예: 팀 실행 순서를 정한 오케스트레이터)
```

확인 방법: `! ls .claude/agents .claude/skills`로 목록을 보거나 "만들어진 에이전트와 스킬의 이름과 역할을 한 줄씩 알려줘"라고 요청한다. 확인한 뒤 `/exit` → `claude`로 다시 시작한다. 다시 시작한 세션에서 `/` 목록에 새 스킬이 보이면 성공이다 확인 필요.

막힐 때

- 하네스가 인식되지 않음(플러그인 없음) → M4 25번(플러그인 설치)을 따른다. `/plugin` 화면에서 설치 목록에 harness가 있는지 본다 확인 필요.
- 계획에 `slides.html`, `workbook/` 같은 앱 파일이 올라옴 → 거부하고 "팀 구성 파일만 만들어줘"를 다시 요청한다.
- 계획에 개인정보 수집 항목이나 외부 AI 키 사용이 보임 → 승인하지 말고 "그 항목은 빼줘"라고 요청한다.
- 다시 시작했는데도 스킬이 `/` 목록에 안 뜸 → `! ls .claude/skills`로 파일이 있는지 확인하고, 있으면 `/reload-skills`를 입력해 본다 확인 필요. 파일이 없으면 하네스 구성을 다시 요청한다.
- 5분 안에 끝나지 않음 → 팀 생성이 끝날 때까지 승인 창만 처리하고, 확인과 재시작은 59번 시작 전에 한다.

해설 · 발표자 노트

소요 시간 5분

오늘의 핵심 요청은 "하네스를 구성해줘" 한 문장과 작업 설명이다. M4에서 퀴즈 팀을 만든 방식과 같다. 하네스는 설명을 읽고 계획을 세운 뒤 팀 파일을 만든다. 플랜 모드 승인은 M2에서 계획을 확인하고 승인한 절차와 같다. Auto 모드로 시작했다면 Shift+Tab을 세 번 눌러야 plan에 닿을 수 있어, 붙여 넣을 내용 맨 앞에 `/plan `을 붙이는 편이 쉽다. 계획에서 사람이 볼 것은 세 가지다. 만들 팀이 ①②③ 작업(슬라이드, 워크북 배포, 검수)을 모두 맡는가, 개인정보·비밀 키·교사용 정답 금지가 반영되었는가, 앱 파일을 아직 만들지 않는가. 팀 파일의 세부 내용(도구 권한, 문구)은 오늘 모두 읽지 않는다. 구성 요소는 처음 나온 용어만 정의한다. 에이전트는 역할이 정해진 하위 작업자 파일, 스킬은 재사용 절차 파일, 오케스트레이터는 팀의 실행 순서를 정한 스킬이다. 재시작하는 이유: 새로 만든 `.claude/agents`, `.claude/skills` 폴더는 세션을 시작할 때 없었으므로, 다시 시작해야 인식된다(근거 https://code.claude.com/docs/en/sub-agents, https://code.claude.com/docs/en/skills). 파일 내용이 마음에 들지 않아도 지금 고치려 하지 않는다. 59번 실행 결과를 보고 말로 고친다. (5분)

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 스킬 이름은 하네스가 정함
- 설치 범위에 따라 새 폴더에서 보이는지
- 상세 없음

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

59

수업 슬라이드 만들기: 하네스 팀 실행 1단계

실습 7분 확인 필요

- 58번에서 다시 시작한 세션에서 생성된 팀을 실행하되, **수업 슬라이드(`slides.html`)까지만** 만들게 한다.
- 구성: 학습 목표 · 도입 질문 · 핵심 개념(여러 장) · 예제 · 확인 문제 · 정리. 8~10장, 키보드(← →)로 넘긴다.
- 파일을 더블클릭해 브라우저로 열고 넘겨 본다.
- 워크북 웹앱과 배포는 63번에서 별도로 요청한다.

프롬프트 · 파일

```
방금 만든 하네스 팀을 실행해줘. 이번에는 unit-spec.md로 수업용 슬라이드 slides.html까지만 만들고, 워크북과 배포는 아직 하지 마.
slides.html은 한 파일로 만들고, 외부 파일이나 외부 주소를 불러오지 마. ← → 키로 슬라이드를 넘기고, 8~10장으로 구성해줘. 마지막 슬라이드에 출처를 표기해줘. unit-spec.md에 없는 교과 내용은 지어내지 말고 "확인 필요"라고 화면에 표시해줘.
```

예상 결과

`unit-project/slides.html`이 생기고, 브라우저에서 열면 8~10장이 순서대로 넘어간다(학습 목표 → 도입 질문 → 핵심 개념 → 예제 → 확인 문제 → 정리, 마지막 장에 출처). 슬라이드 내용이 `unit-spec.md`의 학습 목표·핵심 개념·문항과 겹친다. 슬라이드 개수와 문구는 실행마다 다르다.

막힐 때

- 슬라이드가 10장을 넘거나 8장에 못 미침 → "slides.html을 8~10장으로 맞춰줘. 학습 목표와 정리는 유지해줘"라고 요청한다.
- 키보드로 안 넘어감 → "← → 키로 넘기는 기능을 고쳐줘. 외부 파일은 쓰지 마"라고 요청한다.
- 명세에 없는 내용이 사실처럼 쓰임 → 해당 문장을 붙여 "unit-spec.md에 근거가 없으면 '확인 필요'로 바꿔줘"라고 요청한다.
- 모든 단계를 한꺼번에 실행하려 함 → Esc로 멈추고 "슬라이드까지만 만들어줘"라고 다시 요청한다 확인 필요.
- 팀 실행이 시작되지 않음 → 58번에서 `/exit` → `claude`로 다시 시작했는지 확인한다. 그래도 안 되면 "하네스 팀 없이 Claude Code가 직접 slides.html을 만들어줘"라고 요청해도 된다.

해설 · 발표자 노트

소요 시간 7분

생성된 오케스트레이터 스킬이 팀의 실행 순서를 정한다. 이 호출 방식으로 "슬라이드까지만"이 실제로 지켜지는지는 리허설에서 한 번 확인해 둔다 확인 필요 스킬 이름은 하네스가 정하므로 프롬프트에는 이름 대신 "방금 만든 하네스 팀"이라고 적었다. 슬라이드를 한 파일로 만들고 외부 의존을 없애는 이유는 연수장 인터넷이 불안정해도 열려야 하고, 학생 PC에서 파일 하나만 복사하면 열리기 때문이다. "확인

필요" 표시는 교과 내용을 지어내지 않게 하려는 장치다. 모델은 명세에 없는 사실도 그럴듯하게 쓸 수 있으므로, 표시가 붙은 곳을 교사가 직접 채운다. 7분 중 앞 5분은 생성과 승인, 뒤 2분은 넘겨 보며 학습 목표와 슬라이드가 맞는지 훑는다. 정확한 점검은 다음 장에서 한다. 속도가 느린 참가자는 먼저 생성을 시작하게 하고 기다리는 동안 60번 점검표를 읽게 한다. (7분)

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 단계 분할 동작
- 상세 없음

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

60

슬라이드 60 / 71

사람이 확인: 교과 내용과 학습 목표

점검 3분

- **교과 내용의 최종 확인은 교사가 한다.** 검수 담당의 결과는 참고만 한다.
- ① 내용이 정확한가 ② 슬라이드가 학습 목표 2~3개를 모두 다루는가 ③ 출처·저작권이 표기되었는가 ④ 개인정보가 없는가

프롬프트 · 파일

(선택. 하네스 팀의 검수 담당에게 먼저 읽게 하고 싶을 때)

```
하네스 팀의 검수 담당으로 slides.html이 unit-spec.md의 학습 목표와 맞는지 점검해줘. 학습 목표마다 어느 슬라이드가 다루는지 표로 정리하고, 명세와 다른 내용이나 근거 없는 내용은 슬라이드 번호와 함께 목록으로 알려줘. 파일은 고치지 마.
```

막힐 때

- 점검 결과가 "문제 없음"으로 나옴 → 그래도 ①을 직접 읽어 확인한다. 검수 담당도 틀릴 수 있다.
- 고칠 곳이 나옴 → 고칠 문장과 슬라이드 번호를 정해 Claude Code에 "N번 슬라이드의 이 문장을 이렇게 고쳐줘"라고 요청한다. 고친 뒤 사람이 다시 읽는다.

해설 · 발표자 노트

소요 시간 3분

점검표 네 가지를 직접 눈으로 확인하는 3분이다. ①은 정의·용어·예제의 정확성이고, 교과 전문성이 필요한 부분이라 교사만 판단할 수 있다. ②는 학습 목표 하나하나가 최소 한 장에서 다뤄지는지 대조한다. ③은 명세의 출처 항목이 마지막 장에 반영되었는지, 교과서 그림이나 문장을 그대로 복사하지 않았는지 본다. ④는 슬라이드 예시에 실제 학생·교사 이름이 들어가지 않았는지 본다. 검수 담당의 점검 결과는 참고 자료이고, 고치는 일은 교사의 요청으로 이뤄진다. 선택 프롬프트는 시간이 남는 사람만 쓴다. (3분)

점검 메모

61 터미널에서 한 번 로그인하면 Claude Code가 이어받는다

슬라이드 61 / 71

개념 2분

터미널에서 한 번 로그인해 두면, 그 로그인으로 하던 명령을 Claude Code가 대신 실행합니다.

Firebase: `firebase login` 한 번 → 프로젝트 조회, 연결 값 가져오기, 배포를 Claude Code가 합니다. (다음 슬라이드에서 실습)

GitHub: `gh auth login` 한 번 → "저장소를 만들어 올려줘" 같은 요청을 Claude Code가 `gh` 명령으로 실행합니다. (오늘 실습에서는 쓰지 않습니다)

자동으로 이어지는 것은 명령 실행입니다. 무엇을 실행할지는 내가 승인합니다.

해설 · 발표자 노트

소요 시간 2분

한 문장으로 말해 준다: "터미널에서 한 번 로그인해 두면, 그 계정으로 하던 명령을 Claude Code가 대신 실행하므로 이후 작업이 이어진다. 단, 자동인 것은 실행이고 승인은 사람이 한다." 로그인은 사람이 직접 해야 하는 부분(브라우저에서 계정 선택·허용)이고, 그 뒤의 명령은 Claude Code가 터미널 명령으로 실행한다. 승인 화면은 권한 모드에 따라 달라지므로 "명령마다 승인"을 기본으로 안내하고, 승인 화면에서 배포·올리기(push) 같은 공개 행위 명령은 읽고 승인하게 한다. Firebase: 공식 Firebase 플러그인(Firebase MCP 서버)은 Firebase CLI와 같은 자격증명을 쓴다(근거 https://firebase.google.com/docs/cli/mcp-server). 그래서 `firebase login`을 한 번 해 두면 다음 슬라이드의 프로젝트 조회와 배포를 Claude Code가 이어서 한다. GitHub: 깃허브 CLI(`gh`)는 `gh auth login`으로 브라우저 로그인을 하면 토큰이 컴퓨터의 자격증명 저장소에 보관된다(근거 https://cli.github.com/manual/gh_auth_login). Claude Code 공식 문서는 "create a pr"처럼 요청하면 Claude가 `gh pr create`로 풀 리퀘스트를 만든다고 안내한다(근거 https://code.claude.com/docs/en/common-workflows). `gh`는 따로 설치해야 하므로 오늘은 설치를 안내하지 않고, 원리만 알려 준다. 주의 세 가지: ① 로그인한 계정으로 할 수 있는 일은 Claude Code도 할 수 있으므로 실습에는 연습용 프로젝트·저장소를 쓴다. ② 깃허브에 올리면 공개될 수 있으니 저장소는 비공개로 만들고 비밀 키·학생 정보는 올리지 않는다. ③ 학교 계정과 개인 계정을 섞어 로그인하지 않는다. 원본 대비 변경: 원본에 없는 설명으로, M7 Firebase 연결 실습을 이해하는 배경으로 새로 넣었다. (2분)

도식 설명 가로 4칸 흐름. ① 내가 로그인(터미널에서 한 번만) → ② 컴퓨터에 저장(내 컴퓨터에 로그인 정보가 남음) → ③ Claude Code 실행(같은 로그인으로 명령 실행) → ④ 내가 승인(명령마다 허용 여부 선택). 사람이 하는 칸(①, ④)과 Claude Code가 하는 칸(③)을 색으로 구분한다. 아래에 두 줄 예시: Firebase `firebase login` → 배포, GitHub `gh auth login` → 저장소에 올리기.

62 Firebase 연결: 미리 로그인해 두기

슬라이드 62 / 71

실습 4분 확인 필요

- [ ] **1.** 브라우저에서 Firebase 콘솔(`console.firebase.google.com`)에 구글 계정으로 로그인하고, M6에서 만든 테스트 프로젝트가 보이는지 확인한다. 이 로그인은 계정 확인용이다.
- [ ] **2.** 터미널에서 `firebase login`을 실행하고, 열린 브라우저에서 같은 구글 계정을 골라 허용한다. 한 번만 하면 된다.
- [ ] **3.** 터미널에서 Firebase 플러그인을 설치한다. 이렇게 하면 Claude Code가 이 로그인을 이어받아 프로젝트 조회와 배포를 한다.
- [ ] **4.** Claude Code에서 "내 Firebase 프로젝트 목록을 보여줘"로 연결을 확인한다. 프로젝트 ID가 보이면 성공이다.

프롬프트 · 파일

일반 터미널에서 순서대로 실행한다(Claude Code 밖).

```
firebase login
firebase projects:list
claude plugin marketplace add firebase/firebase-tools
claude plugin install firebase@firebase
```

그다음 Claude Code 입력창에서:

```
내 Firebase 프로젝트 목록을 보여줘
```

예상 결과

`firebase projects:list`에 M6 테스트 프로젝트 ID가 표로 나오고, Claude Code도 같은 프로젝트 목록을 답한다. 이 연결이 끝나면 다음 슬라이드(63)에서 연결 값 가져오기, 규칙 작성, 배포를 Claude Code가 한다.

막힐 때

- `firebase` 명령이 없다고 나옴 → Node.js(20 이상)를 설치한 뒤 `npm install -g firebase-tools`를 실행한다. 학교 PC에서 설치가 막히면 Claude Code에 방법을 물어본다 확인 필요.
- 브라우저가 안 열림 → `firebase login --no-localhost`를 실행하고 화면에 나온 안내대로 코드를 붙여 넣는다.
- Docker 경로(M1 5~7번)로 시작함 → `firebase`는 이미지에 들어 있다. 로그인은 `docker compose run --rm claude firebase login --no-localhost`로 먼저 해 두면 보관되어 이 슬라이드에서는 확인만 하면 된다 확인 필요.
- 프로젝트 목록이 비어 있거나 권한 오류 → 콘솔에 로그인한 계정과 `firebase login`에서 허용한 계정이 같은지 확인한다. 다르면 `firebase logout` 후 다시 로그인한다.
- 학교 관리 계정(Google Workspace)이 관리자 정책으로 막힘 → 개인 구글 계정(테스트 계정)으로 한다 확인 필요.
- `claude plugin` 명령이 안 됨 → 수동 연결로 대신한다: `claude mcp add firebase npx -- -y firebase-tools@latest mcp`, 확인은 `claude mcp list`.

해설 · 발표자 노트

소요 시간 4분

이 슬라이드의 목적은 "Firebase 쪽 설정은 사람이 한 번만 하고, 이후는 Claude Code가 이어받는다"를 체험하는 것이다. 콘솔(브라우저)에 로그인해 두는 것과 Firebase CLI 인증은 따로이며, 공식 문서에서 콘솔 로그인이 CLI 인증을 대신한다는 근거는 찾지 못했다. 그래서 `firebase login`을 한 번 실행해 같은 구글 계정으로 허용하고, 콘솔에 먼저 로그인해 두면 그 계정 선택이 쉽다는 정도로만 안내한다. 브라우저에서 계정 선택 화면이 어떻게 보이는지는 공식 문서에 설명이 없어 확인 필요다. 공식 Firebase 플러그인(Firebase MCP 서버)은 Claude Code에서 프로젝트 조회, 연결 값 가져오기, 보안 규칙 검증, 배포 같은 도구를 쓰게 해 주고, Firebase CLI와 같은 자격증명을 쓴다(근거 https://firebase.google.com/docs/cli/mcp-server). Claude Code 안에서 `!`를 앞에 붙여 `! firebase login`처럼 실행하는 방법은 문서에 브라우저가 열리는 명령의 동작이 명시되어 있지 않으므로 연수에서는 일반 터미널을 쓴다 확인 필요. 4분 중 1~2단계(콘솔 로그인 확인, `firebase login`)는 연수 전에 해 두면 더 여유 있다. 사람이 하는 일은 콘솔 로그인 확인, `firebase login` 허용, 이후 계획·명령 승인이고, 새 콘솔 작업은 없다. 배포는 공개 행위이므로 63번에서 계획을 읽고 승인한다. 보안은 한 문장: 63번에서 Claude Code가 로그인한 본인 uid 문서만 읽고 쓰는 `firestore.rules`를 함께 만들어 배포한다. 교사용 정답은 웹앱 파일에 넣지 않는다(페이지 소스로 보임). 이 구조는 Spark(무료)로 되고 Storage와 Cloud Functions(Blaze 필요)는 쓰지 않는다. 학생 응답은 콘솔 Firestore 데이터 화면에서 확인한다. (4분)

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 학교 PC 설치 권한
- Windows 리허설
- 공식 문서 근거 없음
- 상세 없음

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

63

슬라이드 63 / 71

워크북 웹앱 만들고 배포하기

실습 8분 확인 필요

☐ **1.** 콘솔에서 프로젝트 ID를 확인한다: 톱니 아이콘 > 프로젝트 설정 > 일반. 확인 필요

☐ **2.** 아래 프롬프트의 (프로젝트 ID) 자리에 넣어 붙여 넣는다. 플랜 모드(Shift+Tab)로 시작한다.

☐ **3.** 플랜 모드 계획을 읽는다. 무엇을 배포하는지 한국어로 설명되어 있는지 확인하고 승인한다.

☐ **4.** 62번에서 Firebase 연결을 마쳤으므로 로그인은 이미 되어 있다. 로그인 오류가 나오면 62번 막힐 때를 따른다.

☐ **5.** 배포가 끝나면 출력된 배포 주소를 확인한다.

프롬프트 · 파일

(프롬프트의 (프로젝트 ID)를 1단계에서 확인한 값으로 바꾼다.)

```
하네스 팀으로 워크북 웹앱 단계만 실행해줘. slides.html은 다시 만들지 마.

내 Firebase 프로젝트 ID는 (프로젝트 ID)야. unit-spec.md의 문항으로 학생 워크북 웹앱을 workbook/ 폴더에
만들고 Firebase Hosting으로 배포해줘.
- 문항은 로그인 없이 보이게 하고, Google 로그인은 답을 저장하고 다시 열어 불러올 때만 써줘.
- Firebase 연결 값은 firebase CLI로 네가 가져와줘. 안 되면 나에게 알려 줘.
- firestore.rules(로그인한 본인 uid 문서만 읽기·쓰기)도 함께 만들어 배포해줘.
- 교사용 정답은 웹앱 파일에 넣지 마. 개인정보와 비밀 키는 저장하거나 코드에 넣지 마.
- 배포 전에 무엇을 배포하는지 한국어로 설명해줘.
- 도구 설치와 명령 실행은 실행 전에 나에게 물어봐.
- 배포가 끝나면 배포 주소를 알려줘.
```

예상 결과

`unit-project/` 안에 `workbook/` 폴더와 `firestore.rules`, `firebase.json`이 생기고, 배포가 끝나면 터미널에 배포 주소가 출력된다. 파일 이름과 구조는 실행마다 다를 수 있다.

막힐 때

- Node.js나 firebase CLI가 없다고 나옴 → Claude Code에 "설치 방법을 알려주고 설치는 내가 승인한 뒤 실행해줘"라고 요청한다 확인 필요.
- 로그인 관련 오류 → 62번 막힐 때(`firebase login --no-localhost`, 계정 확인, `firebase logout` 후 재로그인)를 따른다.
- 프로젝트가 없다고 나오거나 권한 오류 → 콘솔의 프로젝트 ID와 `firebase login`에 쓴 계정이 같은지 확인한다.
- Firestore 데이터베이스가 없다고 나옴 → M6 43번 순서로 만든다.
- 기본 Hosting 사이트가 없다는 오류 → 2026-10-15부터 신규 프로젝트는 기본 사이트가 자동 생성되지 않는다(근거 https://firebase.google.com/docs/hosting/default-site). 오류 문구를 Claude Code에 그대로 전달해 사이트 생성 방법을 묻는다 확인 필요.
- 프로젝트가 없는 참가자(M6를 따라오지 못함) → 짝 화면으로 함께 보고, 59-60번 슬라이드 완성에 집중한다. 프로젝트는 M6 38번부터 만들 수 있다.

해설 · 발표자 노트

소요 시간 8분

8분 중 1분은 프로젝트 ID 확인, 나머지는 계획 확인과 배포 대기다. 연결 설정을 연수 전에 해 두면 더 여유 있다. 이 실습에서 사람이 하는 일은 프로젝트 ID 전달과 승인뿐이다(로그인은 62번에서 마쳤다). 프롬프트가 짧은 이유: 파일 구조, 문서 경로, SDK 버전은 Claude Code가 정하게 두고, 목적(로그인 없이 문항, 저장·불러오기, 배포)과 금지(정답·개인정보·비밀 키)만 말한다. 연결 값은 `firebase apps:sdkconfig`로 가져오게 하며 확인 필요, 안 되면 프로젝트 ID만으로 진행할 방법을 Claude Code에게 묻는다. 팀 단계 분할이 동작하지 않으면 "하네스 팀으로" 문장을 빼고 Claude Code에 직접 요청해도 결과는 같다. 배포는 공개 행위이므로 계획에서 배포 대상(웹앱 폴더와 규칙)을 읽고 승인한다. 시간이 걸리면 승인 대기를 강사가 확인하고, 56번 도식으로 돌아가 사람이 할 일을 다시 짚는다. (8분)

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- 한국어 UI 이름
- Node.js 필요 여부
- 상세 없음
- 명령 동작

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

---

64 배포 주소로 확인하기

슬라이드 64 / 71

실습 4분 확인 필요

- 새 탭에서 배포 주소(`https://프로젝트ID.web.app` 형태)를 연다.
- 확인 5가지: ① 문항이 로그인 없이 보임 ② Google 로그인(테스트 계정) ③ 답 저장 ④ 새로고침 후 불러오기 ⑤ (선택) 콘솔 Firestore 데이터에서 저장된 문서 확인.
- 확인이 끝나면 주소를 짝에게 전한다.

막힐 때

- 로그인 팝업에서 `auth/unauthorized-domain`이 뜸 → 오류 문구를 Claude Code에 그대로 보여 주면 콘솔 보안(Security) > Authentication > 설정(Settings) 탭 > 승인된 도메인(Authorized domains)에 어떤 도메인을 추가할지 알려 준다. 사람이 콘솔에서 그 도메인만 추가한다 확인 필요.
- 화면이 비어 있음/404 → 오류 문구나 화면을 그대로 Claude Code에 전달한다.
- 저장 시 `Missing or insufficient permissions` → 오류 문구를 그대로 Claude Code에 전달한다.

해설 · 발표자 노트

소요 시간 4분

4분 동안 배포된 결과를 직접 확인한다. 로그인 없이 문항이 보여야 짝이 주소만 열어서 볼 수 있다. 로그인은 저장과 불러오기에만 쓴다. ⑤는 시간이 남는 사람만 한다. 콘솔 Firestore 데이터 화면에서 응답이 저장된 문서를 볼 수 있다. 로그인 팝업 오류 `auth/unauthorized-domain`은 M6 41번에서 한 번 보았다. 승인된 도메인은 로그인 팝업을 허용하는 주소 목록이다. 2025-04-28 이후 만든 프로젝트는 기본 목록이 다를 수 있어, Hosting 기본 주소의 포함 여부는 오류가 날 때만 확인한다 확인 필요. 오류가 나면 문구를 그대로 Claude Code에 전달하는 습관을 65번에서 연습한다. (4분)

이 쪽의 확인 필요 항목 (연수 전 강사가 확인)

- Hosting 기본 주소가 기본 포함인지
- 상세 없음

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

슬라이드 65 / 71

65 고쳐 달라고 말하고 다시 배포하기

실습 5분

☐ **1.** 안내 문구나 문항 안내를 한 줄 바꿔 달라고 요청한다.

☐ **2.** 계획(무엇을 바꾸고 배포하는지)을 읽고 승인한다.

☐ **3.** 같은 주소를 새로고침해 바뀐 문구를 확인한다. 다시 배포해도 주소는 같다.

- 오류가 나면 화면이나 콘솔의 문구를 그대로 Claude Code에 붙여 넣는다. 이 "오류 되돌려주기"가 오늘의 핵심 습관이다.
- 선택 심화: 교사용 열람 화면을 요청해 본다.

프롬프트 · 파일

```
학생 화면 맨 위의 안내 문구를 "답을 쓰고 저장 버튼을 누르세요"로 바꿔줘. 바꿀 내용을 먼저 설명하고, 같은 주소로 다시 배포해줘. 명령 실행은 내가 승인한 뒤에 해.
```

예상 결과

Claude Code가 바꿀 내용을 설명하고 승인을 요청한다. 배포가 끝난 뒤 같은 주소를 새로고침하면 새 문구가 보인다. 반영이 늦으면 다시 새로고침한다.

막힐 때

- 새로고침해도 문구가 그대로 → 배포가 끝났는지 확인하고, 안 바뀌면 화면 상태를 그대로 Claude Code에 전달한다.
- 오류 문구가 나옴 → 문구 전체를 복사해 Claude Code에 붙여 넣고 "이 오류를 고쳐줘"라고 요청한다.
- 배포 명령에서 권한 오류 → 63번 막힐 때(계정과 프로젝트 ID 확인)를 따른다.

해설 · 발표자 노트

소요 시간 5분

5분 동안 "말로 고치고 다시 배포하는" 순환을 체험한다. 사람이 하는 일은 한 줄 요청과 승인이다. 배포는 같은 주소를 갱신하므로 짝에게 전한 주소가 바뀌지 않는다. 핵심은 오류 되돌려주기다. 화면이나 콘솔에 나온 오류 문구를 사람이 해석하려 하지 않고 그대로 Claude Code에 붙여 넣으면, 원인을 찾아 고치는 일을 Claude Code가 한다. 선택 심화인 교사용 열람 화면은 시간이 부족하면 연수 후 과제로 둔다. (5분)

☐ 직접 해 보았다 ☐ 예상 결과와 같았다 ☐ 막혀서 도움을 받았다

실습 메모

66

슬라이드 66 / 71

짝과 바꿔 점검하기

점검 3분

- 짝이 알려 준 배포 주소를 **내 브라우저**로 열어 4가지를 확인한다.
- ① 교과 내용·문항이 단원 학습 목표와 맞는가 ② 로그인 없이 문항이 보이고 안내 문구가 이해되는가 ③ 글자 크기·접근성 ④ 개인정보와 정답이 페이지 소스에 없는가(오른쪽 클릭 > 페이지 소스 보기).
- 남의 프로젝트에 내 계정으로 로그인하지 않는다. 문항과 안내를 보는 데까지만 한다.
- 고칠 점은 짝이 자기 Claude Code에 요청한다.

막힐 때

- 주소가 열리지 않음 → 짝에게 주소를 다시 받고, 짝의 배포가 끝났는지 확인한다.
- 페이지 소스가 읽기 어려움 → 정답이나 이름처럼 보이는 단어를 검색(Ctrl+F)해 본다. 판단이 어려우면 짝이 자기 Claude Code에 "이 웹앱 파일에 교사용 정답이나 개인정보가 있는지 알려줘"라고 요청한다.
- 고칠 점이 나옴 → 짝이 자기 Claude Code에 요청하고, 고친 뒤 같은 주소에서 둘이 다시 확인한다.

해설 · 발표자 노트

소요 시간 3분

서로의 화면을 보며 3분 동안 점검표를 채운다. 칭찬 하나, 고칠 점 하나를 말하게 한다. ①은 60번에서 본인이 본 것을 짝의 눈으로 다시 보는 것이다. 본인이 놓친 교과 내용 오류를 다른 교사가 발견하는 경우가 많다. ④는 이 프로젝트의 핵심 점검이다. 웹앱 파일은 누구나 페이지 소스로 볼 수 있으므로, 교사용 정답이나 개인정보가 거기에 있으면 공개된 것이다. 남의 프로젝트에 내 계정으로 로그인하지 않는 이유: 짝 프로젝트에 내 계정의 응답을 남기지 않기 위해서다. 테스트 계정 = Firebase 실습에 쓰는 개인 구글 계정(학생·업무 계정이 아닌 것). 점검이 늘어지면 ①과 ④를 먼저 하게 한다. (3분)

점검 메모

67 슬라이드 67 / 71

오늘 만든 패키지 한 장 회고

정리 2분

- 만든 것 · 막힌 곳 · 다음 단원에 적용할 점을 한 장에 쓴다.
- 공유: 배포 주소를 전하거나 `unit-project` 폴더를 압축한다. 공유 전에 개인정보와 비밀 키가 없는지 다시 본다.
- 테스트 프로젝트 삭제는 M6 마지막 슬라이드(55번)를 따른다. 삭제하면 배포한 워크북 웹앱도 더는 동작하지 않으므로, 계속 쓰려면 삭제하지 않는다.

도식 · 단계

1. 만든 것: `unit-spec.md` · 하네스 팀(`.claude/`) · `slides.html` · `workbook/` · `firestore.rules` (체크칸)
2. 막힌 곳과 푼 방법: _____
3. 다음 단원에 적용할 점: _____

아래 띠 "공유 방법": 배포 주소 전달 / 폴더 압축 / 둘 다 하지 않음 중 선택 칸. 맨 아래 한 줄 "공유 전 확인: 개인정보 없음 · 비밀 키 없음".

해설 · 발표자 노트

소요 시간 2분

한 줄씩만 쓰게 한다. "오늘 만든 것은 앱 한 개가 아니라, 내 교과 단원 하나를 수업 패키지로 만드는 절차입니다." 다음 단원에는 `unit-spec.md` 만 새로 쓰고 하네스 팀을 다시 실행하면 같은 팀이 쓰인다. 서비스 계정 키 파일이나 AI 키가 폴더에 있다면 공유하지 않는다. 연수가 끝나면 M6 55번의 안내대로 테스트 프로젝트를 삭제할 수 있다. 삭제하면 배포한 워크북 웹앱도 더는 동작하지 않으므로, 계속 쓰려면 삭제하지 않는다. 이 슬라이드는 실습 시간으로 센다. 다음은 마무리(68번)다. (2분)

도식 설명 회고 한 장 카드. 위쪽 3칸 가로 배치. 칸 제목과 빈 줄:

MODULE MW

마무리

마무리

시간	10분
슬라이드	68-71 (4장)

이 모듈의 슬라이드

번호	제목	유형	시간	쪽
68	오늘 해낸 일 체크리스트	정리	3분	·
69	흔한 안티패턴 2가지	정리	2분	·
70	자주 묻는 질문 4가지	정리	3분	·
71	참고 자료와 다음 걸음	정리	2분	·

68 오늘 해낸 일 체크리스트

슬라이드 68 / 71

정리 3분

- 나는 이제 이것을 할 수 있다 (손으로 체크)
- ☐ Claude Code로 실습 폴더에서 대화했다
- ☐ CLAUDE.md에 규칙을 등록했다
- ☐ 스킬(/quiz)을 만들어 재사용했다
- ☐ 하네스로 팀 파일을 만들었다
- ☐ 하네스가 만든 팀 파일(에이전트·스킬)을 열어 보고 말로 고쳤다
- ☐ 우리 교과 단원 수업 패키지(슬라이드와 학생 워크북)를 만들어 Firebase Hosting 주소로 확인했다

예상 결과

참가자가 6칸 중 몇 개를 체크했는지 스스로 센다. 체크하지 못한 칸은 연수 후 배포 자료로 다시 해 보면 된다.

> **막힐 때**
>
> 체크 못 한 칸이 많아도 괜찮다. "못 한 것 하나만 골라 다음 주에 해 보기"로 정하면 된다.

해설 · 발표자 노트

소요 시간 3분

(대본) "자, 마지막으로 오늘 한 일을 스스로 점검해 봅시다. 화면의 여섯 가지 중 해낸 것에 체크해 보세요. 못 한 것이 있어도 괜찮습니다. 배포 자료가 그대로 남아 있습니다." 30초 체크, 짝에게 가장 어려웠던 칸 말하기 1분, 정리 1분 30초.

- **전체 체크리스트 10항목(1-8번은 원본 HTML 체크리스트 11항목 중 이 덱에서 다루는 8항목을 원래 순서대로, 9·10번은 이 덱에서 추가한 Firebase 항목이라 HTML에는 없다):**
  1. Claude Code를 설치하고 실습 폴더에서 최소 한 번 이상 대화해 봤다.
  2. CLAUDE.md에 우리 업무에 맞는 규칙을 등록해 봤다.
  3. 나만의 스킬(슬래시 커맨드)을 최소 하나 만들어 재사용해 봤다.
  4. "모델이 아니라 하네스가 결과를 결정한다"는 명제를 내 언어로 설명할 수 있다.
  5. `revfactory/harness` 플러그인을 설치하고, 자연어 요청 한 번으로 에이전트 팀 파일이 만들어지는 것을 직접 확인했다.
  6. 하네스가 만든 팀 파일(에이전트·스킬)을 열어 보고, 말로 고치고 다시 실행해 결과를 비교했다.
  7. 하네스로 내 업무용 팀 하나를 만들어 요청해 봤다.
  8. 우리 교과 단원 수업 패키지(슬라이드와 학생 워크북)를 만들어 Firebase Hosting 주소로 확인했고(M7), 다음 주에 실제로 적용해 볼 업무 하나를 정했다.
  9. (추가) Firebase 6개 항목(Authentication, Firestore, 요금제, Storage, Hosting, Cloud Functions)을 언제 쓰는지 말할 수 있다.
  10. (추가) 테스트 프로젝트에서 인증·DB를 설정해 봤다.
- 본문의 6칸은 1, 2, 3, 5, 6, 8번을 줄인 것이다(본문 4칸째가 5번, 5칸째가 6번). 4번과 7번은 본문에서 뺐다. 플러그인 설치가 막힌 참가자는 4·5칸이 비어 있을 수 있다. 강사는 "플러그인이 안 된 분은 4번째 칸이 비어 있어도 됩니다"라고 말한다.
- 원본 대비 변경: 원본의 "2.12절" 표기를 "M7"로 바꿨고, 5~8번은 하네스 활용 중심 구성(M4~M7)에 맞게 문장을 바꿨고, 1번의 "클로드 코드"는 덱 표기에 맞춰 "Claude Code"로 썼다. 9·10번은 설계서 지시로 추가. (3분)

도식 설명 6칸 체크박스 카드(2열 3행). 오른쪽 아래에 작은 글씨로 "나머지 4개는 배포 자료(노트의 10항목 목록)에" 표시. 체크 칸은 손으로 표시하는 빈 네모.

69

슬라이드 69 / 71

흔한 안티패턴 2가지

정리 2분

- 처음부터 큰 팀을 요청하기 → 역할 1~2개의 작은 팀으로 시작하고 필요할 때 말로 추가 요청
- 만들어진 파일을 열어 보지 않고 쓰기 → 파일을 읽고 내 교과 기준과 맞는지 확인한 뒤 말로 고치기

예상 결과

참가자가 자기 M7 하네스 요청 프롬프트와 생성된 팀 파일을 다시 보고 2개 중 해당되는 것이 있는지 확인한다(30초).

> **막힐 때**
>
> 하네스가 이미 큰 팀을 만들었다면 "역할을 줄여 달라"고 요청하고, 줄인 팀에서 꼭 필요한 역할만 남았는지 짝과 이야기해 본다.

해설 · 발표자 노트

소요 시간 2분

(대본) "하네스를 처음 쓸 때 빠지기 쉬운 함정이 두 가지 있습니다. 오늘 만든 팀을 떠올리며 들어 보세요. 첫째, 처음부터 큰 팀을 요청하는 것입니다. 역할 한두 개로 시작하고, 부족하면 말로 추가를 요청하세요. 둘째, 하네스가 만든 파일을 열어 보지 않고 그대로 쓰는 것입니다. 파일에는 역할과 검수 기준이 적혀 있으므로 내 교과 기준과 맞는지 읽어 보고, 다르면 말로 고쳐 달라고 하면 됩니다."

- 보충: 하네스가 팀을 필요 이상으로 크게 만들면 "줄여 달라"고 요청한다.
- 연결: M5에서 연습한 '파일 열어 보기, 말로 고치기'와 M7의 하네스 요청·생성된 팀 확인 단계와 같은 습관이다.
- 원본 대비 변경: 카드 본문의 긴 설명을 처방 한 줄로 줄이고 원본의 절 번호(2.2, 2.11, 2.5) 참조는 뺐다. 안티패턴 문장은 직접 설계 가정을 하네스 활용(요청 규모, 생성 파일 확인) 기준으로 바꿨다. (2분)

도식 설명 가로 2장 카드(`.card.bad`). 카드마다 위쪽에 "안티패턴" 태그, 제목 한 줄(위 두 문장의 앞부분), 아래에 처방 한 줄(화살표 뒤 문장). 강조색은 `#b0402c` .

70

슬라이드 70 / 71

자주 묻는 질문 4가지

정리 3분

- 사용료: 무료 계정은 안 되고 유료 Pro 이상이나 Console(API) 계정 필요. 학교 단위 도입은 행정 부서와 상의
- 학생 사용: 연수는 교사용 기준. 학교 AI 지침·개인정보 방침을 먼저 검토
- 다른 교사와 공유: CLAUDE.md·스킬·하네스가 만든 에이전트 파일은 평범한 텍스트 파일 → 공유 드라이브나 Git으로 공유
- Firebase 요금 걱정: 연수에서는 Blaze로 올렸다. 예산 알림은 알림일 뿐 사용을 막지 않으니, 연수 후 테스트 프로젝트를 삭제하고 청구 내역을 확인한다

예상 결과

참가자가 가장 궁금한 질문 하나를 손들거나 질문 칸에 적는다.

> **막힐 때**
>
> 위 네 개 말고 다른 질문은 노트의 "나머지 FAQ"를 보고 답하거나, 설문에 적게 한다.

해설 · 발표자 노트

소요 시간 3분

(대본) "자주 받는 질문 네 가지만 짚겠습니다. 첫째, 사용료. Claude Code는 무료 claude.ai 계정으로는 쓸 수 없고 Pro, Max, Team, Enterprise 같은 유료 요금제나 Console(API) 계정이 필요합니다. 혼자 시작한다면 Pro로 오늘 실습은 모두 할 수 있었습니다. 여럿이 쓰려면 학교 차원의 Team이나 Enterprise를 행정 부서와 상의하세요. 둘째, 학생이 직접 쓰게 해도 되느냐. 이 연수는 교사의 수업 자료·행정 업무용으로 설계됐습니다. 학생에게 쓰게 하려면 학교의 AI 도구 지침, 개인정보 처리 방침, 계정 관리 방식을 먼저 검토해야 하고, 권한 모드를 더 엄격히 적용하고 사람이 직접 확인하는 단계를 더 두어야 합니다. 셋째, 다른 교사와의 공유. 이 파일들은 모두 폴더 안의 텍스트 파일이라 그대로 공유됩니다. 같은 폴더를 내려받아 실행하면 같은 구성이 적용됩니다. 넷째, Firebase 요금 걱정. 오늘은 Storage와 Cloud Functions를 함께 해 보려고 테스트 프로젝트를 Blaze로 올렸습니다. 무료 사용량 안에서는 비용이 0일 수 있지만 결제 수단은 등록되어 있습니다. 예산 알림은 알려 주기만 할 뿐 사용을 자동으로 멈추지 않습니다. 그래서 연수가 끝나면 테스트 프로젝트를 삭제하고, 다음 달까지 청구 내역과 예산 알림 이메일을 확인하세요. 학교 업무에 계속 쓰려면 결제 수단 등록은 학교 예산 승인이 필요한 사항으로 보세요."

- **Firebase 근거(사실표 8절·요약):** Spark = No-cost, Blaze = Pay as you go. Blaze가 필요한 것은 Cloud Functions 배포와 Cloud Storage(2026-02-03부터). 연수 당일 pricing 페이지로 다시 확인한다. 예산 알림은 비용을 막지 못하고 알려 주기만 한다. 출처: https://firebase.google.com/pricing , https://firebase.google.com/docs/projects/billing/avoid-surprise-bills
- **나머지 FAQ(질문 오면 사용):**
  - 하네스가 만든 팀이 크면 속도·비용이 얼마나 늘어나나요? → 호출할 때마다 별도 대화(컨텍스트)가 새로 열려 호출 횟수만큼 시간과 비용이 늘어난다. 그래서 작은 팀으로 시작하고 필요할 때만 늘려 달라고 요청한다.
  - `revfactory/harness`는 Anthropic 공식 기능인가요? 어떻게 쓰나요? → 아니다. Anthropic이 제공하는 것은 플러그인 마켓플레이스라는 설치 통로이고, harness는 커뮤니티의 서드파티 플러그인이다(v2는 실험 플래그 없이 동작한다). 설치 전 저장소 내용을 확인한다. 쓰는 법은 M4에서 한 대로 설치 후 "하네스를 구성해줘"와 작업 설명을 말하고, 플랜 모드 계획을 확인·승인하면 `.claude/agents/`와 `.claude/skills/`에 파일이 만들어진다. 새로 만들어진 `.claude` 폴더는 재시작해야 인식될 수 있다. 참고로 Claude Code에는 "에이전트 팀"이라는 별도의 실험 기능(`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS`)이 있으나 이 연수에서는 켜지 않는다. 켜 두면 서브에이전트가 팀원으로 실행되어 실습 화면이 달라질 수 있다. 에이전트 팀은 대화형 세션에서만 동작하고 헤드리스(`-p`)에서는 쓸 수 없다.
  - 더 깊이 공부하려면? → 다음 슬라이드(71)의 참고 자료. 특히 책의 Part 4(실전편)가 코드 리뷰 자동화, 풀스택 기능 구현, 레거시 마이그레이션, 디버깅/RCA 사례를 다룬다.
- 원본 대비 변경: 원본 FAQ 답변을 한두 문장으로 줄였다. 원본 FAQ에 없는 Firebase 질문(4번)을 설계서 지시로 추가했다. 원본 답변의 "Pro 요금제로 충분" 표현은 유지했다. (3분)

도식 설명 질문 4칸 카드. 4번째 카드(Firebase)만 보조색(`#356e64`)으로 구분. 4번째 카드에 작은 글씨로 "Blaze가 필요한 것: Cloud Functions 배포, Storage(2026-02-03부터)".

---

71 슬라이드 71 / 71

참고 자료와 다음 걸음

정리 2분

- 도서: 황민호(로빈 황), 《하네스 엔지니어링 with 클로드 코드》, 한빛미디어
- 오픈소스: `revfactory/harness`, `revfactory/claude-code-harness` · 공식 문서: Claude Code, Firebase

다음 걸음: 다음 주에 적용해 볼 업무 하나 + 설문

설문 링크/QR: 강사가 연수 전 삽입

예상 결과

참가자가 다음 주에 적용할 업무 하나를 말하고(또는 적고) 설문에 응답한다.

> **막힐 때**
>
> QR이 안 읽히면 강사가 짧은 주소를 칠판이나 채팅에 적어 준다.

해설 · 발표자 노트

소요 시간 2분

(대본) "참고 자료입니다. 오늘 하네스 개념은 황민호 저자의 책에서 왔습니다. 오픈소스 두 개와 공식 문서도 같이 적어 두었습니다. 마지막으로 다음 주에 실제로 적용해 볼 업무 하나를 정해 보세요. M7에서 쓴 단원 명세(unit-spec.md)와 하네스 요청 문장을 다음 단원에도 그대로 가져다 쓸 수 있습니다. 설문에 남겨 주시면 다음 연수를 고치는 데 쓰겠습니다. 고생 많으셨습니다." 연수 후 Firebase 테스트 프로젝트를 삭제하는 방법은 슬라이드 55에 있다고 한 번 더 알려 준다.

- **링크(원본 그대로, 슬라이드에는 짧은 이름으로, 노트·자료 HTML에는 전체 URL):**
  - 도서: https://www.hanbit.co.kr/books/%ED%95%98%EB%84%A4%EC%8A%A4-%EC%97%94%EC%A7%80%EB%8B%88%EC%96%B4%EB%A7%81-with-%ED%81%B4%EB%A1%9C%EB%93%9C-%EC%BD%94%EB%93%9C?code=B2817272480
  - `revfactory/harness`: https://github.com/revfactory/harness (설치: `/plugin marketplace add revfactory/harness` → `/plugin install harness@harness-marketplace`)
  - `revfactory/claude-code-harness`: https://github.com/revfactory/claude-code-harness (구조화된 사전 설정이 코딩 에이전트 성능에 미치는 영향을 실증한 연구 저장소)
  - Claude Code 공식 문서: https://code.claude.com/docs/en/overview (설치, 퀵스타트, 서브에이전트, 스킬, 슬래시 커맨드)
  - Firebase 공식 문서(조사자 사실표 출처): 시작 https://firebase.google.com/docs/web/setup · 인증 https://firebase.google.com/docs/auth/web/google-signin · Firestore https://firebase.google.com/docs/firestore/quickstart · Storage https://firebase.google.com/docs/storage/web/start · Hosting https://firebase.google.com/docs/hosting/quickstart · Cloud Functions https://firebase.google.com/docs/functions/get-started · 요금 https://firebase.google.com/pricing · 예기치 않은 청구 방지 https://firebase.google.com/docs/projects/billing/avoid-surprise-bills
- 원본 대비 변경: 원본의 "2.2절에서 설치·실습한" 같은 절 번호 표현을 뺐다. Firebase 공식 문서 링크는 원본에 없어 사실표에서 가져와 추가했다. 원본의 "2026년 9월 기준" 문장을 유지했다. 원본의 제작 푸터("Claude(Cowork)로 제작, 도서 인용은 요약·인용이며 세부 내용은 원서 참고")는 슬라이드 맨 아래 작은 글씨 한 줄로 넣어도 좋다. 이 마지막 슬라이드에는 마무리 모듈 종료를 알리는 "수고하셨습니다" 문구를 넣는다. (2분)

도식 설명 왼쪽에 자료 4종 목록(`.refs` 스타일, 종류 태그: 도서/오픈소스/공식 문서), 오른쪽에 "다음 걸음" 카드(위 3번째 줄)와 설문 QR 자리 표시 사각형. 아래 작은 글씨: "2026년 9월 기준 정보입니다. 달라졌다면 /help 와 공식 문서로 확인하세요."